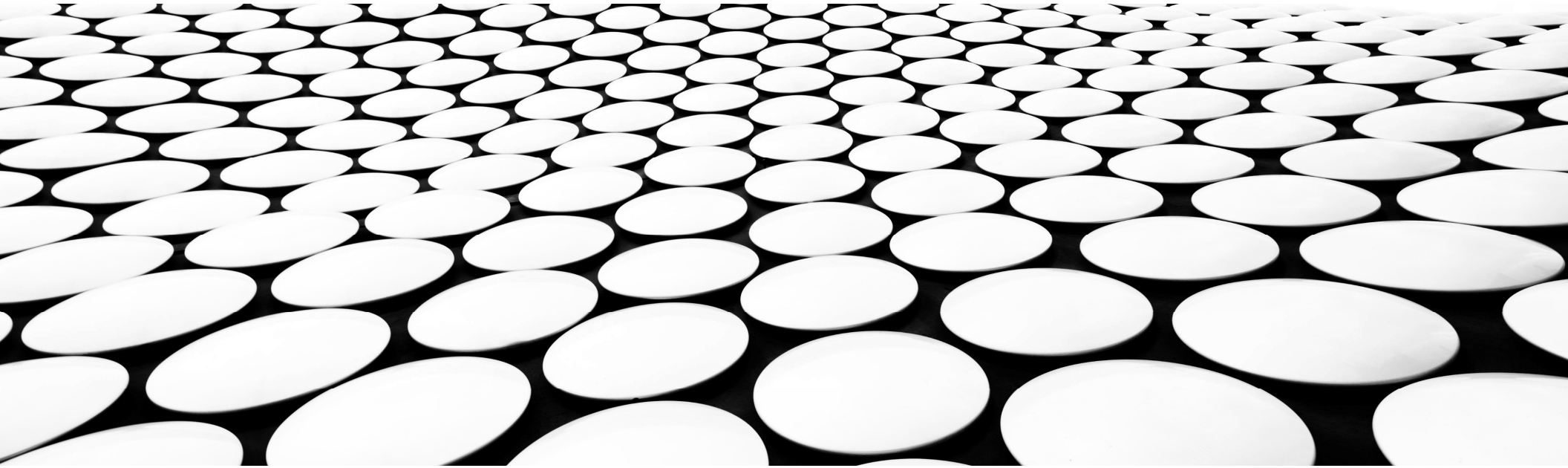
Artificial Neural Networks and Deep Learning



References

Parts of this lecture are based on:

1. Slides from the Harvard Course CS109A Introduction to Data Science by Pavlos Protopapas, Kevin Rader and Chris Tanner, available at https://github.com/Harvard-IACS/2019-CS109A

2. Sharda, R., Dursun Delen, and E. Turban. "Analytics, data science." *& Artificial Intelligence: Systems for Decision Support. Eleventh ed: Pearson* (2019).

Agenda

- Artificial neural network (ANN)
- Deep Learning (DL)

"**AI** is sometimes heralded as the new industrial revolution. If **deep learning** is the steam engine of this revolution, then **data** is its coal: the raw material that powers our intelligent machines, without which nothing would be possible."

—François Chollet, AI researcher

Neural Network Concepts

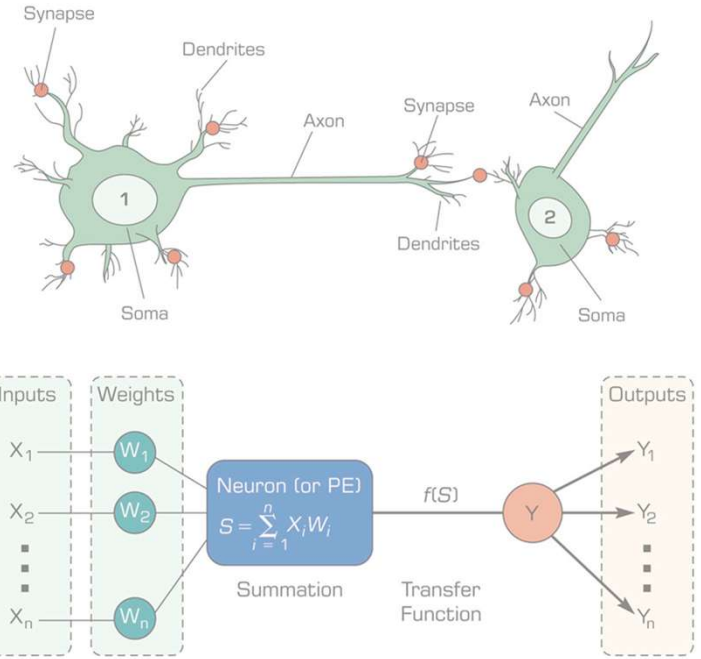
- **Neural networks** (NN): a human brain metaphor for information processing
- **Artificial neural network** (ANN)

- Many uses for ANN for:
  - pattern recognition
  - forecasting
  - prediction and
  - classification

- Many application areas:
  - finance
  - marketing
  - manufacturing
  - operations
  - information systems, and so on

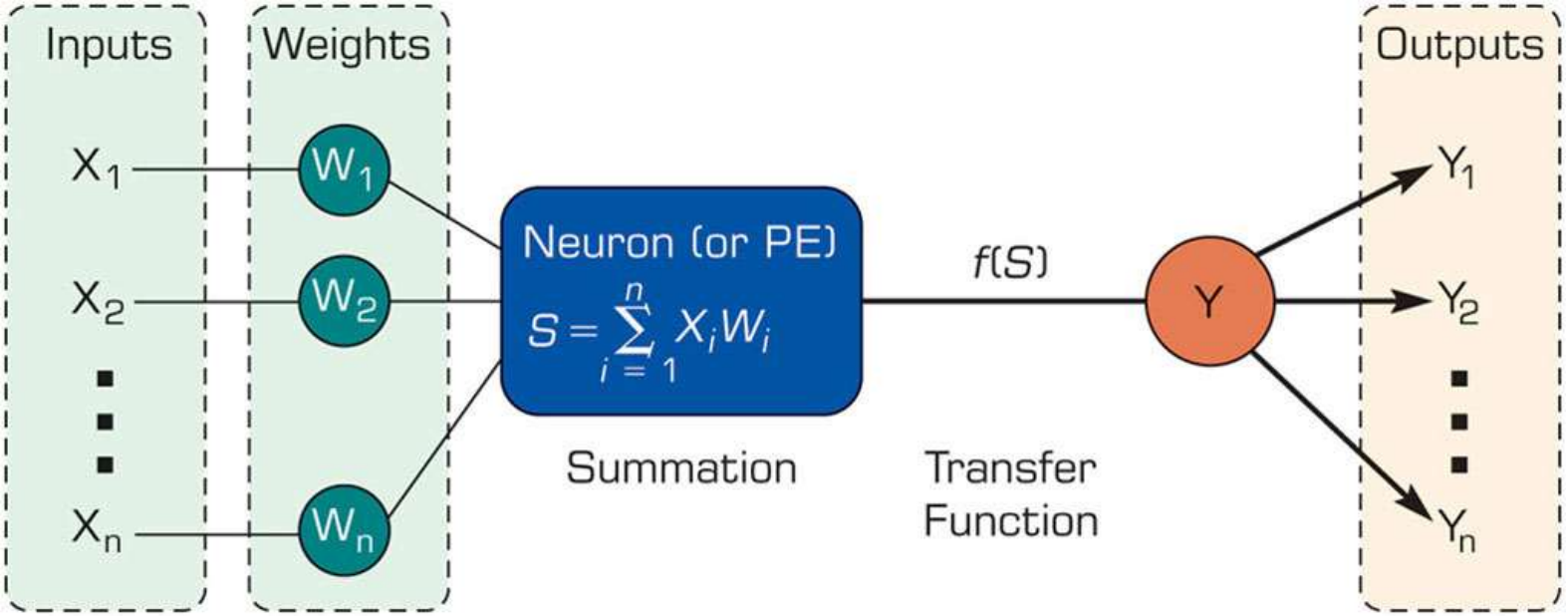
Analogy between Neural Networks and Artificial Neural Networks

Biological	Artificial
Soma	Node
Dendrites	Input
Axon	Output
Synapse	Weight
Slow	**Fast**

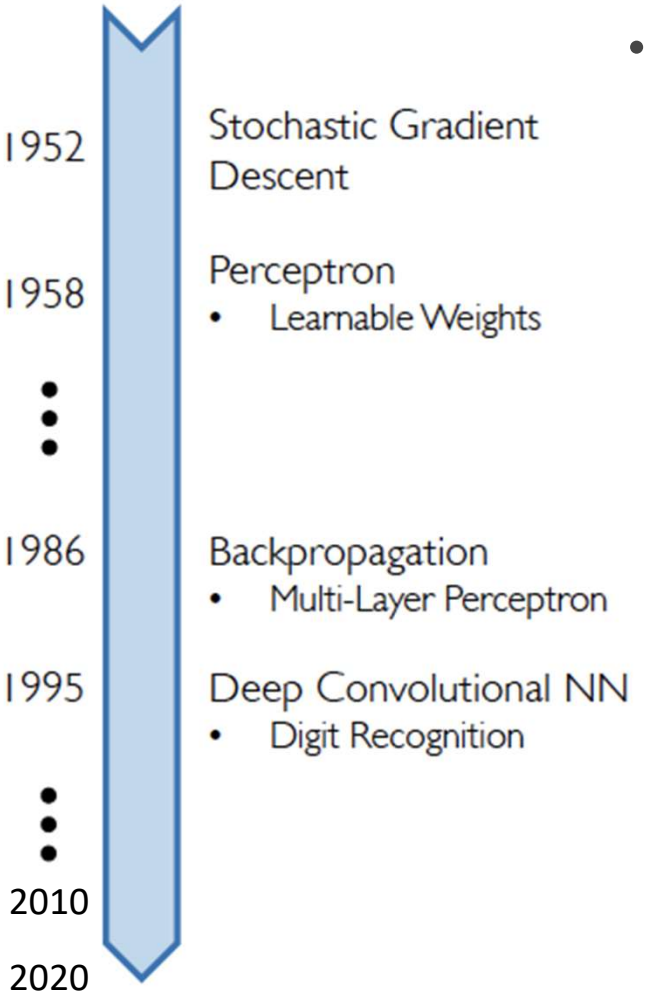


Artificial Neural Network

- A single neuron (processing element – PE) with inputs and outputs



Why Now?



- Neural Networks date back decades, so why the resurgence?

1. Big Data

- Larger Datasets
- Easier Collection & Storage



2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable

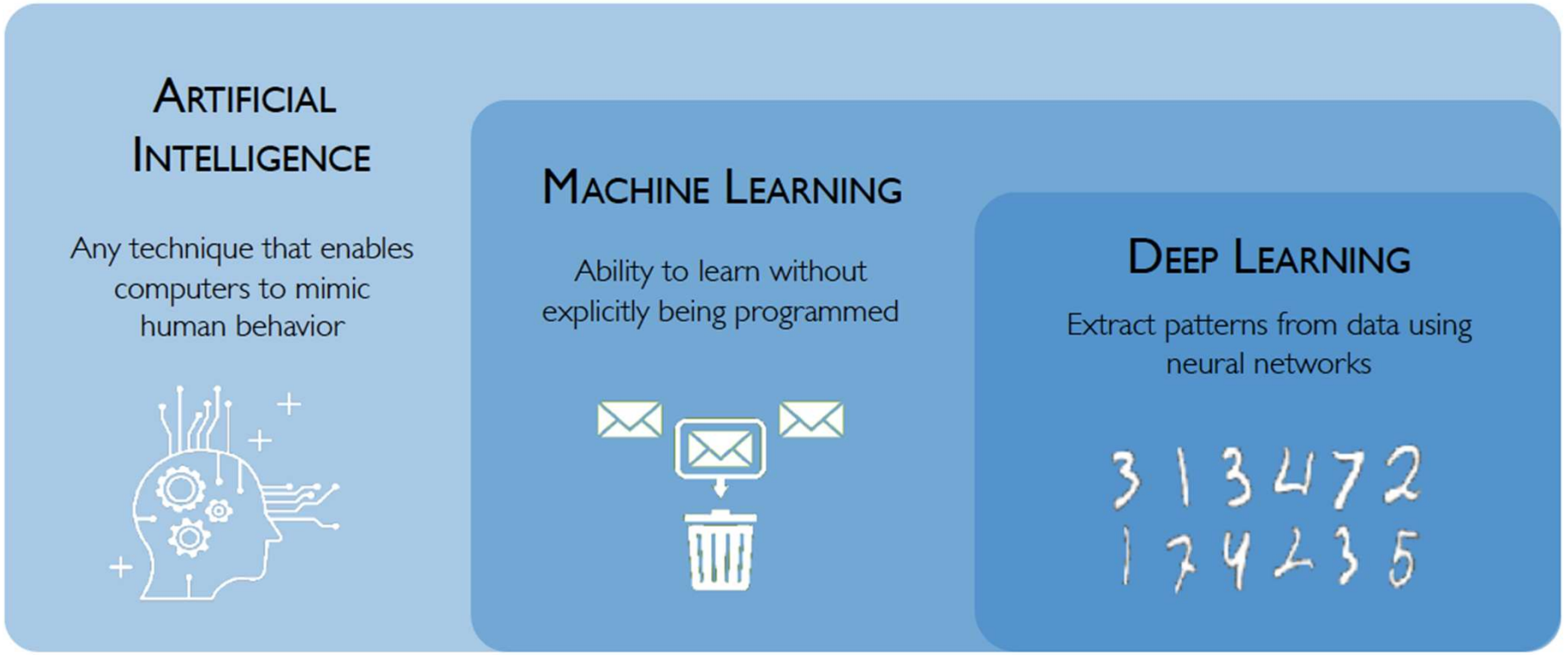


3. Software

- Improved Techniques
- New Models Toolboxes

TensorFlow

What is Deep Learning

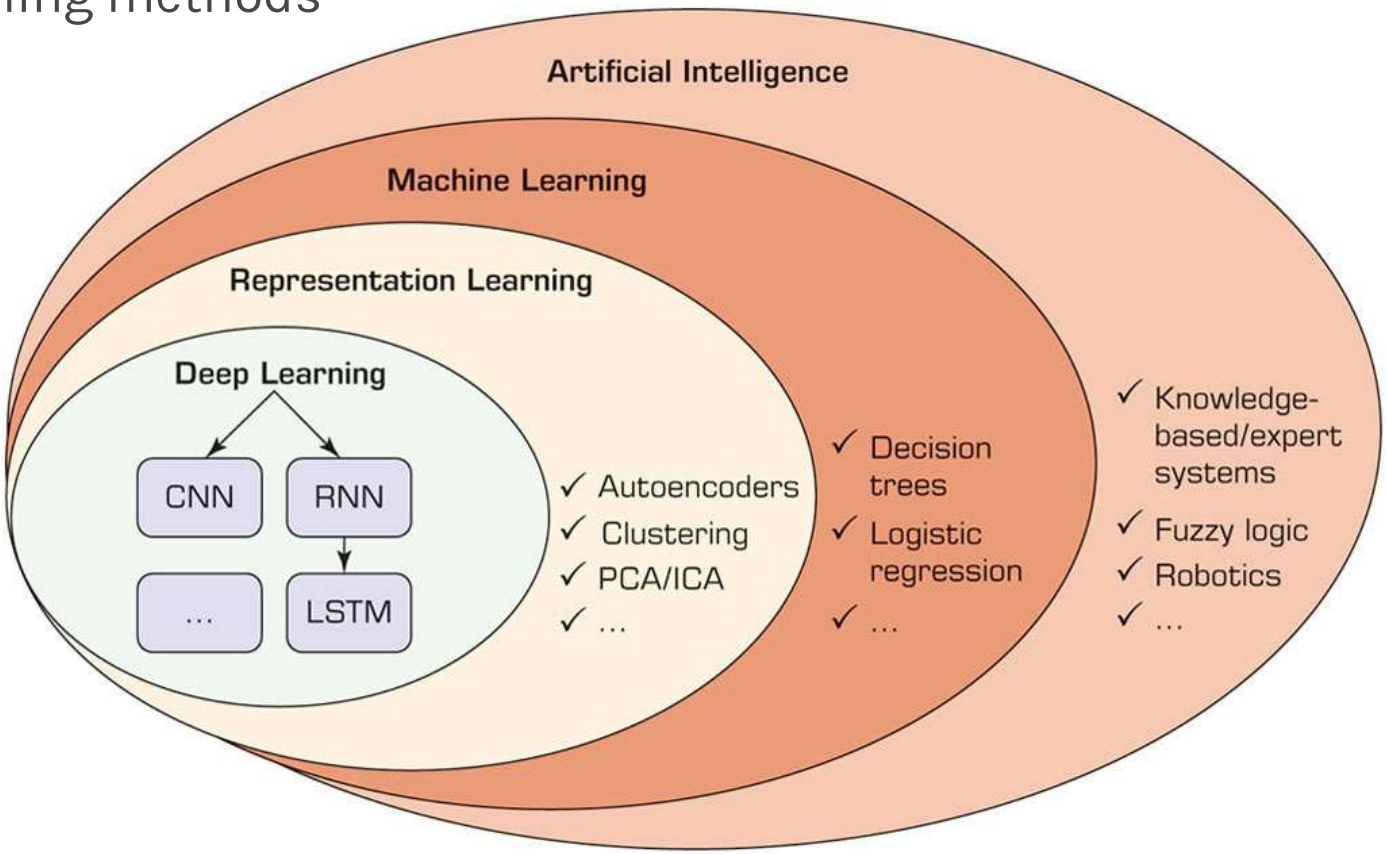


Introduction to Deep Learning

- Neural networks, with their remarkable ability to **derive meaning** from complicated or imprecise data, can be used to **extract patterns** and **detect trends** that are too complex to be noticed by either humans or other computer techniques
  - Adaptive learning
    - adjust their internal parameters (weights) based on experience
  - Self Organization
    - Networks automatically discover relevant features and structure in data
  - Fault Tolerance
    - Robust to noisy, incomplete, or corrupted input data

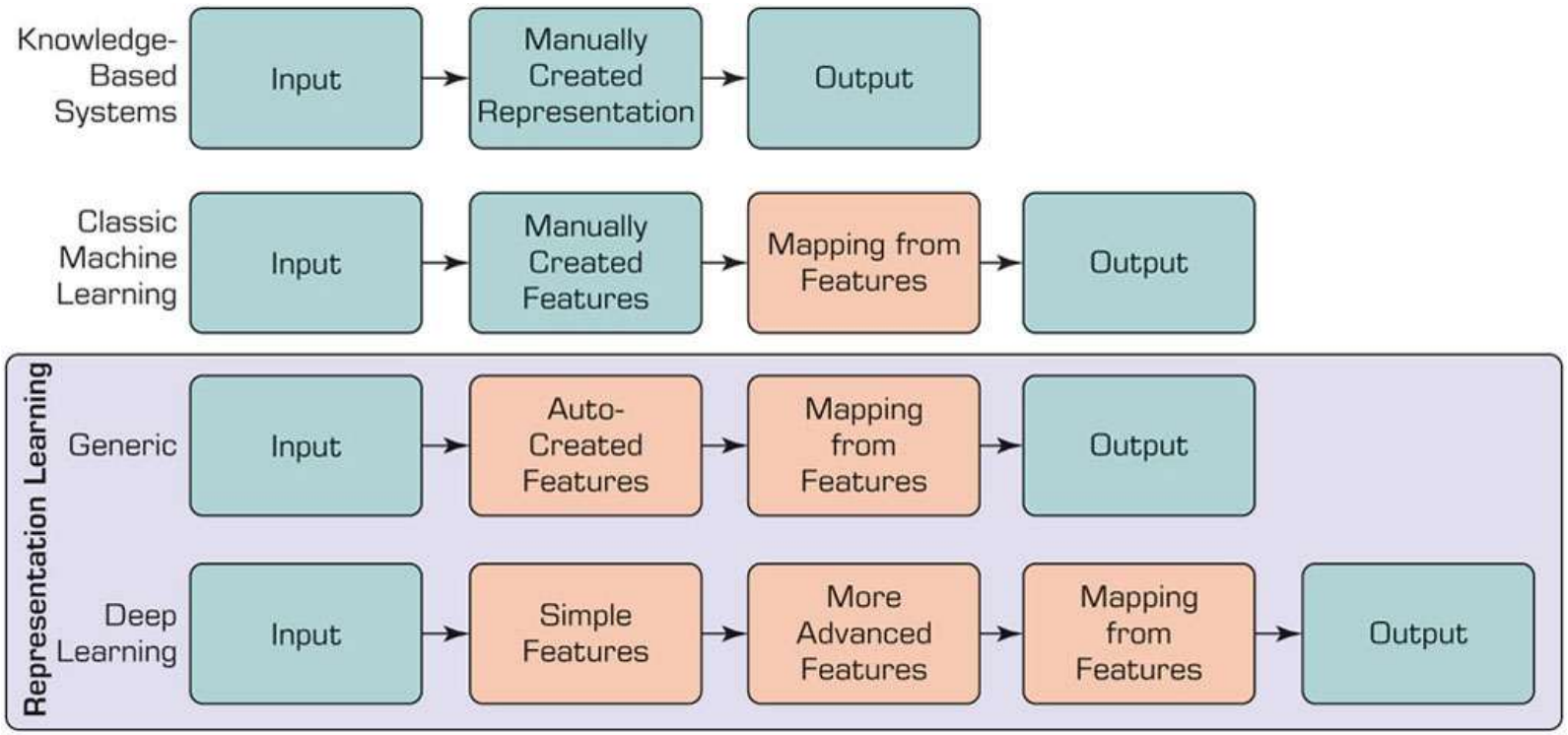
Introduction to Deep Learning

- The placement of Deep Learning within the overarching AI-based learning methods



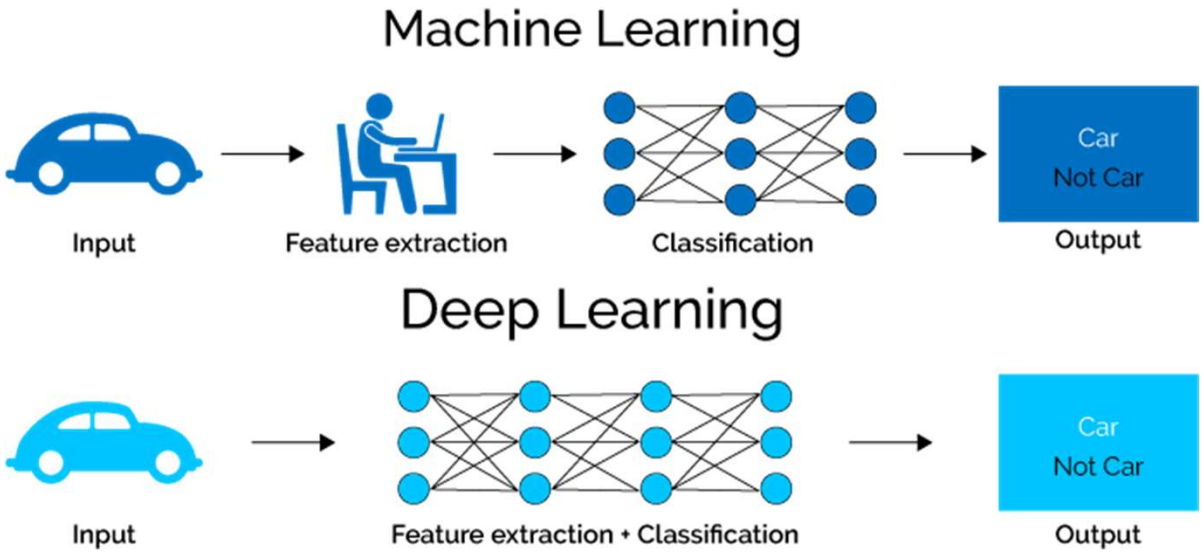
Introduction to Deep Learning

- Differences between Classic Machine-Learning Methods and Representation Learning/Deep Learning

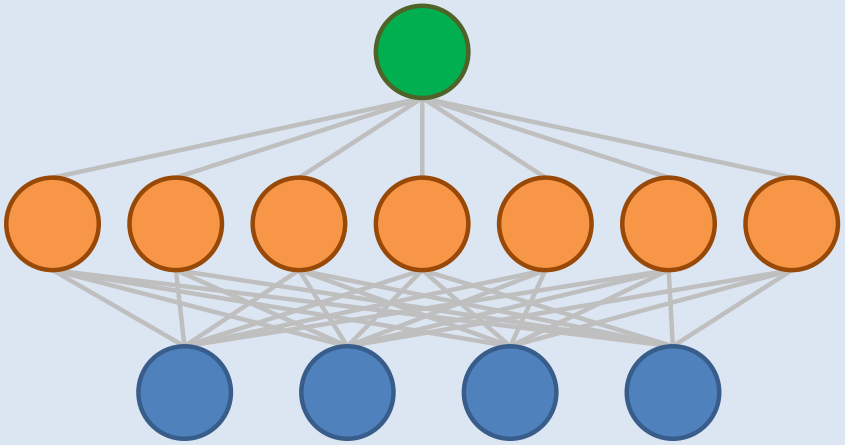


What is Deep Learning (DL) ?

- A machine learning subfield of learning representations of data. Exceptional effective at **learning patterns**.
- Deep learning algorithms attempt to learn (multiple levels of) representation by using a **hierarchy of multiple layers.**
- If you provide the system tons of information, it begins to understand it and respond in useful ways.

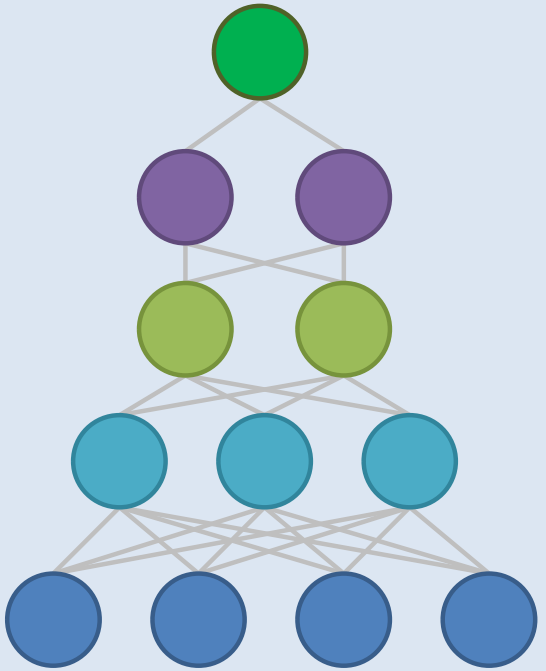


Shallow Network



Shallow (2 layer) networks need a lot more hidden layer nodes to compensate for lack of expressivity

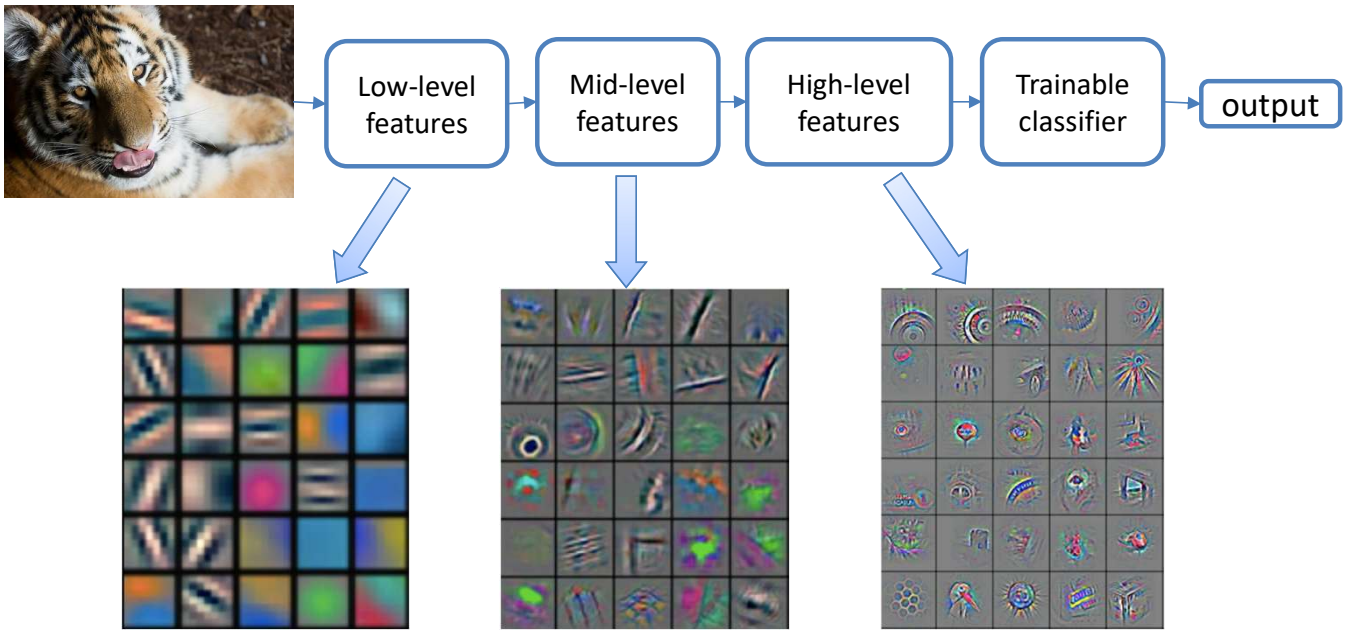
Deep Network



In a deep network, high levels can express combinations between features learned at lower levels

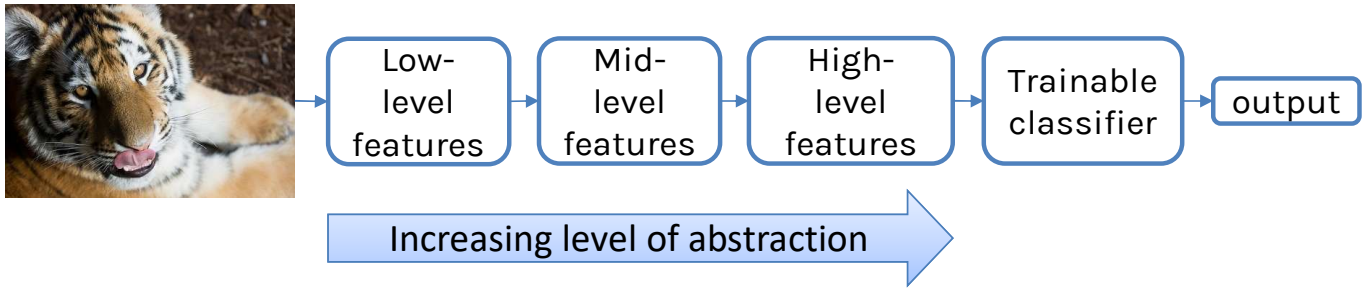
What is Deep Learning (DL) ?

- Deep learning (a.k.a. representation learning) seeks to learn rich **hierarchical representations** (i.e. features) automatically through multiple stage of feature learning process.



Feature visualization of convolutional net trained on ImageNet (Zeiler and Fergus, 2013)

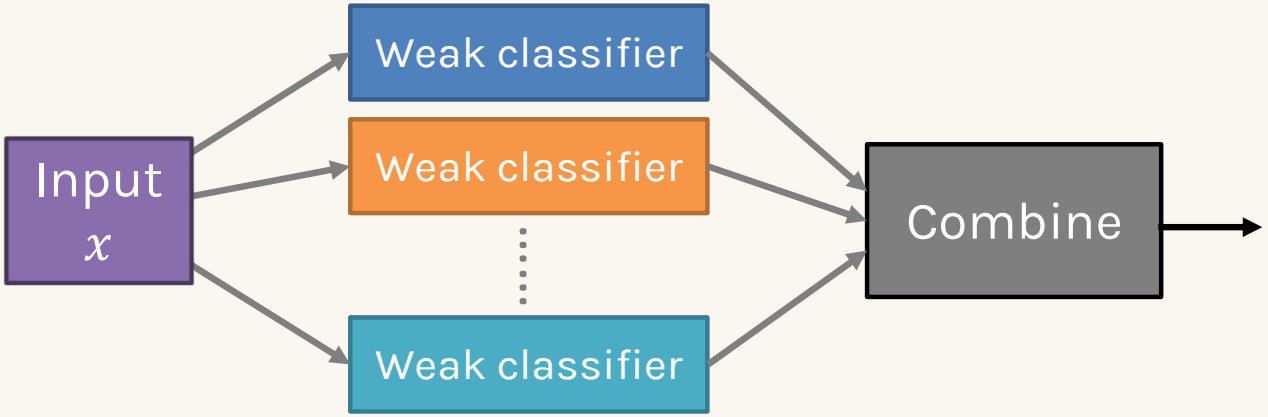
Learning Hierarchical Representations



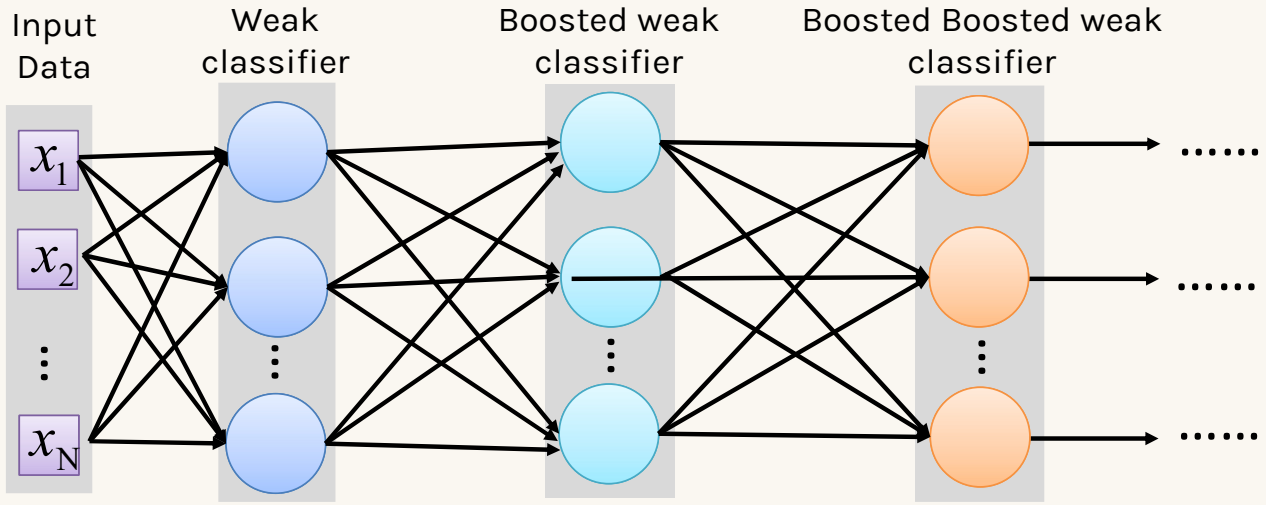
- Hierarchy of representations with **increasing level of abstraction**.
- Each stage is a kind of trainable nonlinear feature transform.
- **Image recognition**
  - Pixel → **edge** → **texton** → **motif** → **part** → object
- **Text recognition**
  - Character → **word** → **word group** → **clause** → **sentence** → story

Boosting vs Deep Learning

Boosting

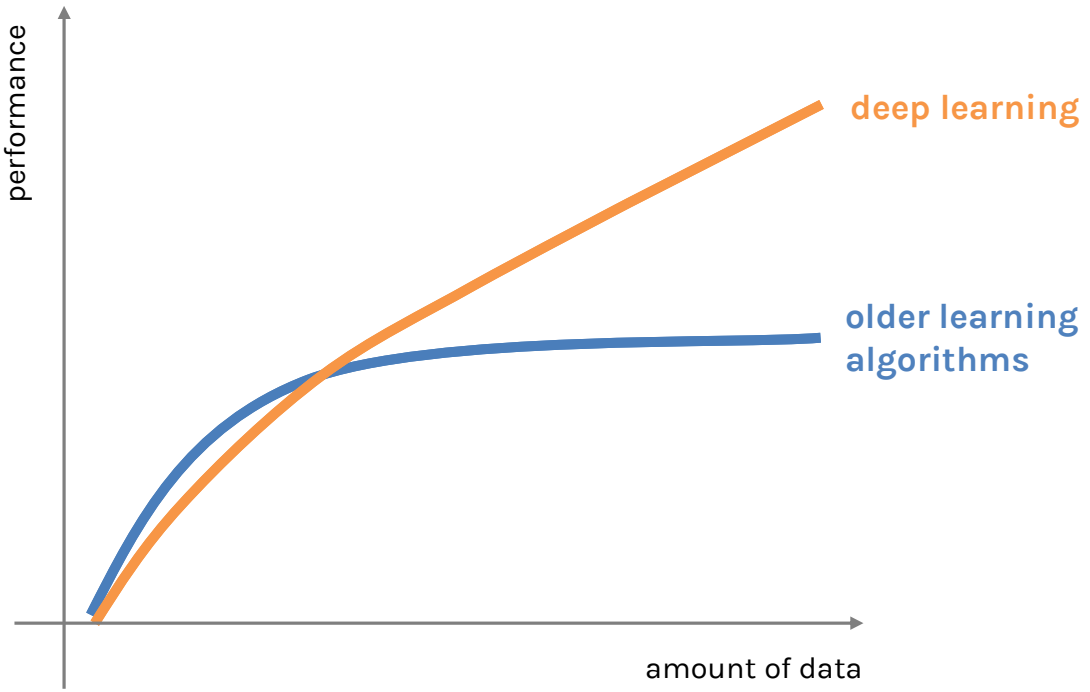


Deep Learning



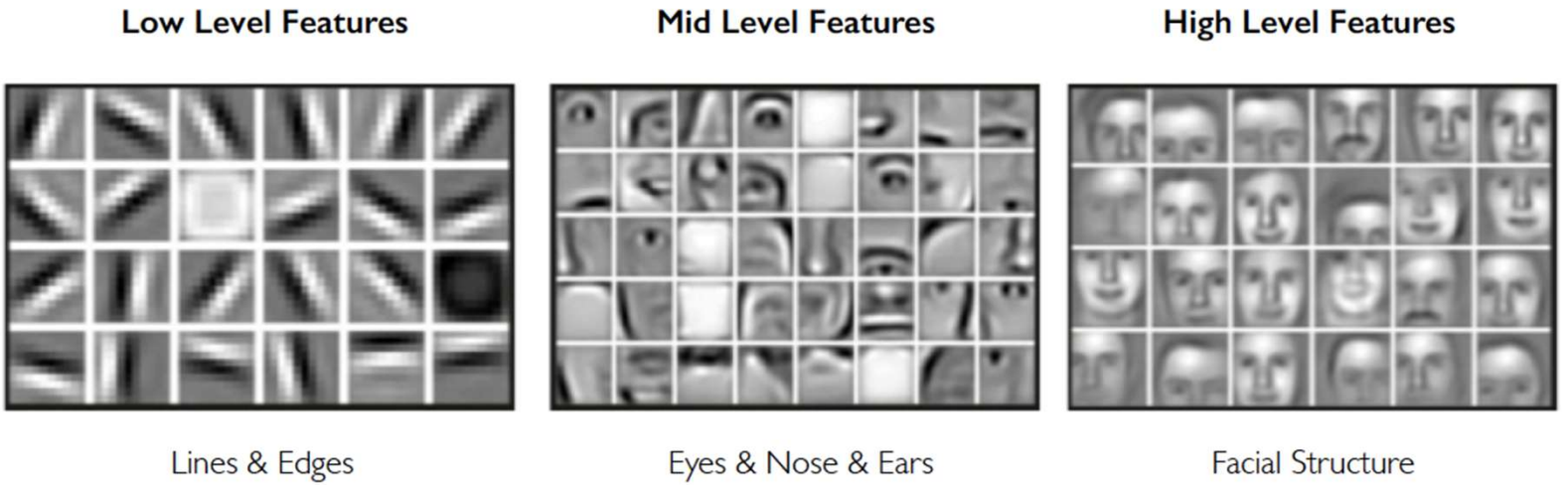
Why Deep Learning?

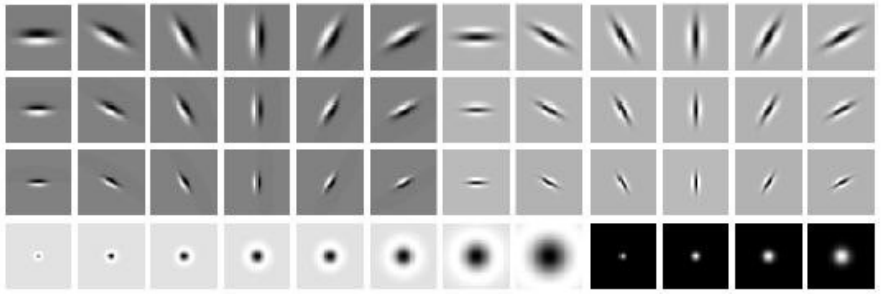
- Performance comparison of Deep Learning and traditional Machine Learning
- Deep learning need a tons of data to create good prediction model



Why is DL useful?

- Hand engineered features are time consuming, brittle and not scalable in practice
- **Can we learn the underlying features directly from data?**



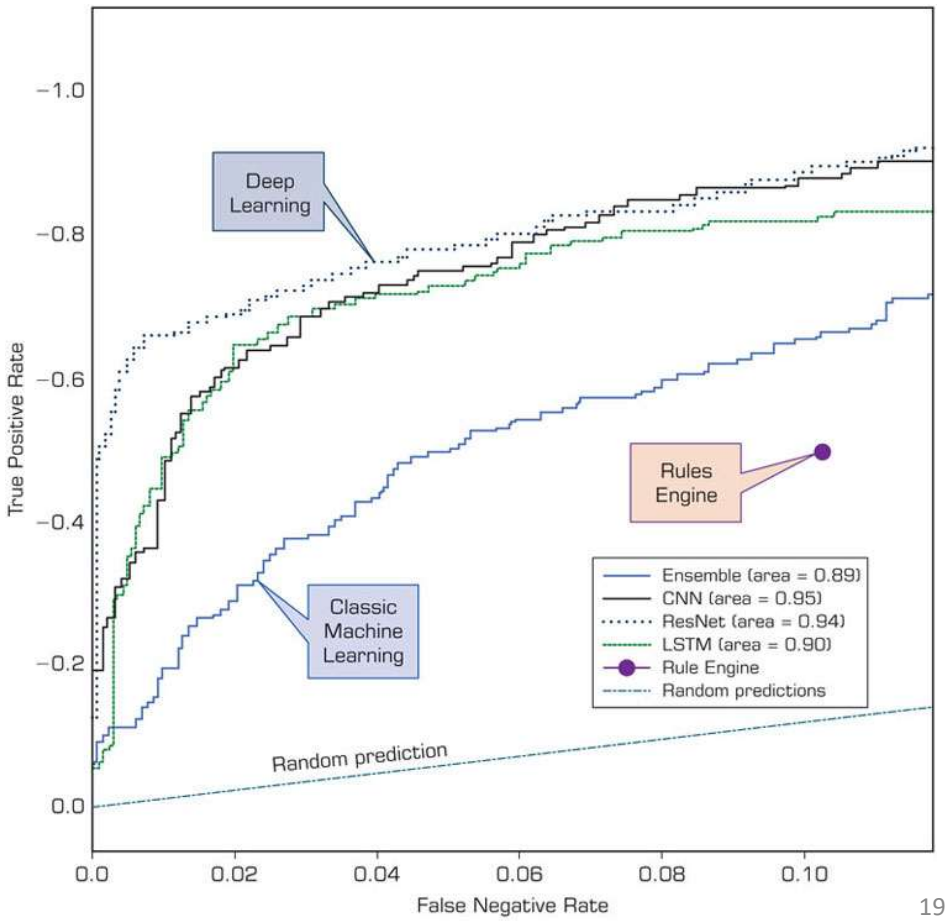


Other mathematical hand-made low-level features

Example: Fighting Fraud with Deep Learning

- Business problem
  - Danske Bank
  - Predictive analytics in banking
    - Fraud detection
- The solution
  - Deep learning

- The results
  - Accuracy
    - ROC curve
  - DL vs traditional ML techniques



Why is DL useful?

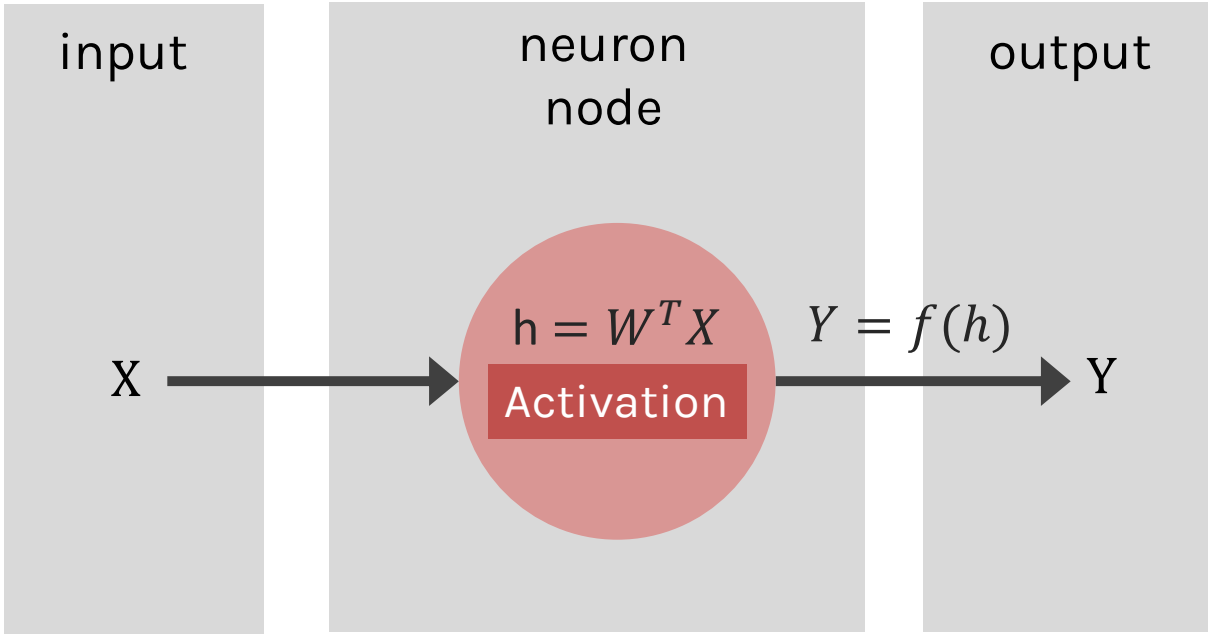
- **Manually designed features** are often **over-specified**, **incomplete** and take a **long time** to design and validate
- **Learned Features** are easy to adapt, fast to learn
- Deep learning provides a very flexible, (almost?) universal, learnable framework for representing world, visual and linguistic information
  - Can learn both unsupervised and supervised
  - Effective end-to-end joint system learning
  - Utilize large amounts of training data

ML competitions

- In 2016 and 2017, **Kaggle** was dominated by two approaches:
  - **Gradient boosting machines -** (XGBoost) is used for problems where structured data is available.
  - **Deep learning -** (Keras) is used for perceptual problems such as image classification.

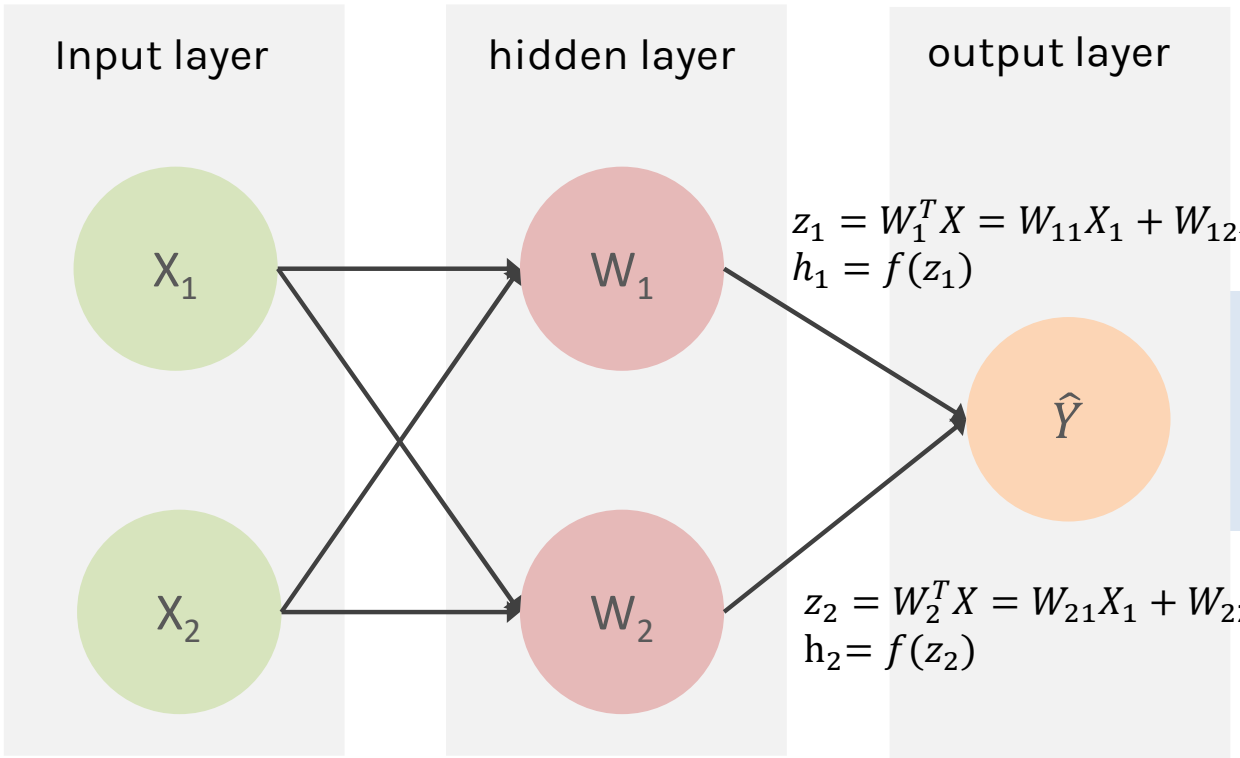


Anatomy of artificial neural network (ANN)



$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_d \end{bmatrix} \qquad W = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_d \end{bmatrix} \qquad Y = f\left(W_0 + \sum_{i=1}^{d} W_i \cdot X_i \right)$$

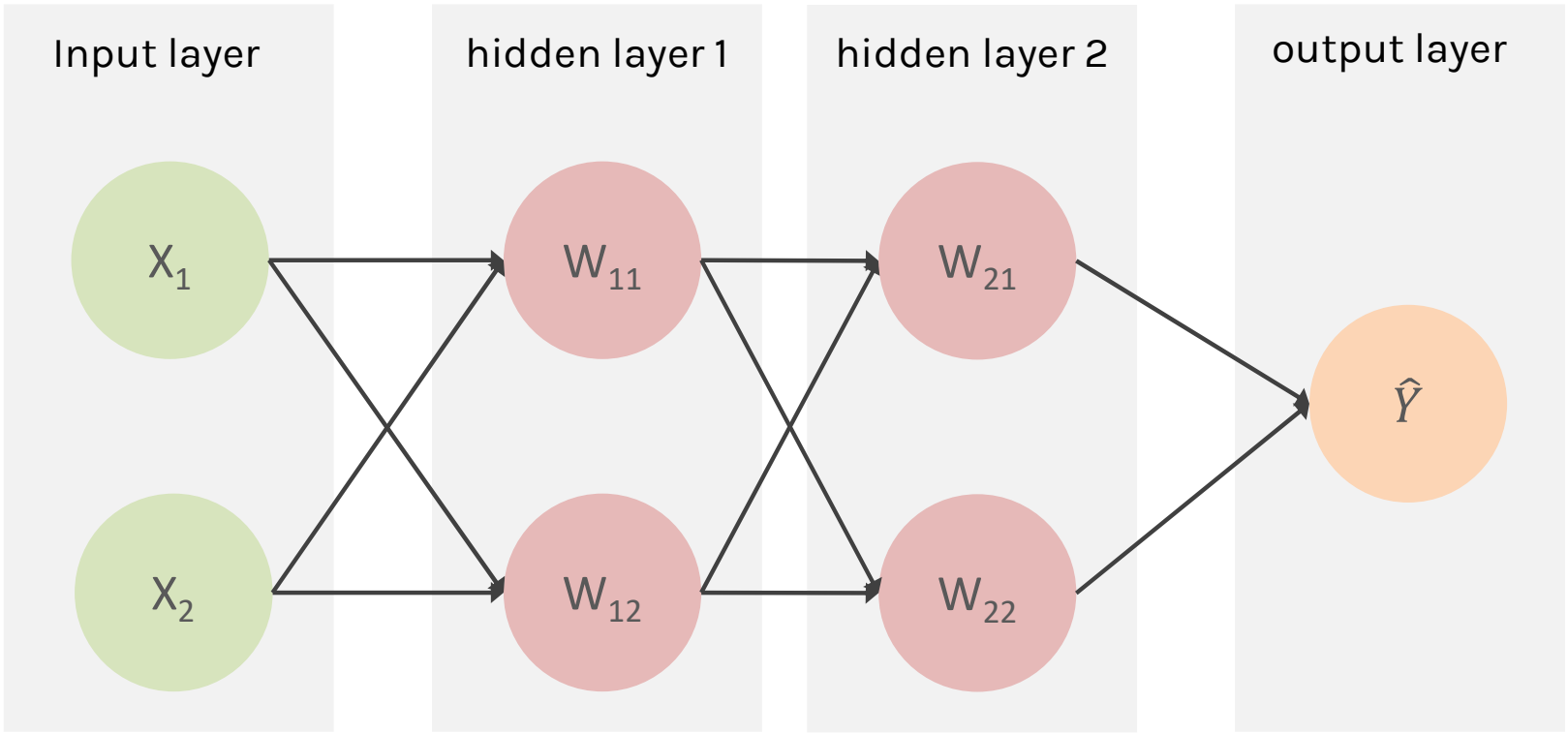
Anatomy of artificial neural network (ANN)



$\hat{Y} = g(h_1, h_2)$ Output function

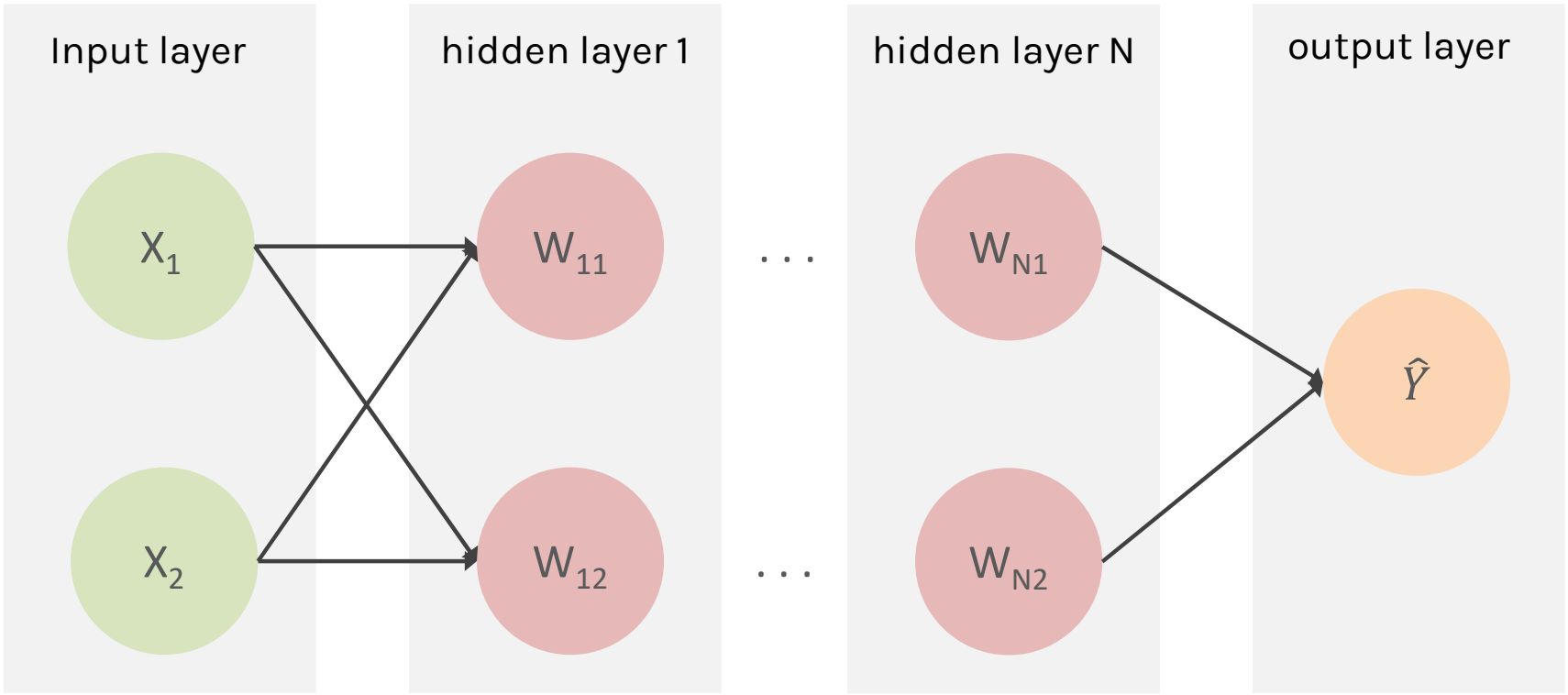
$J = \mathcal{L}(\hat{Y}, Y)$ Loss function

Anatomy of artificial neural network (ANN)



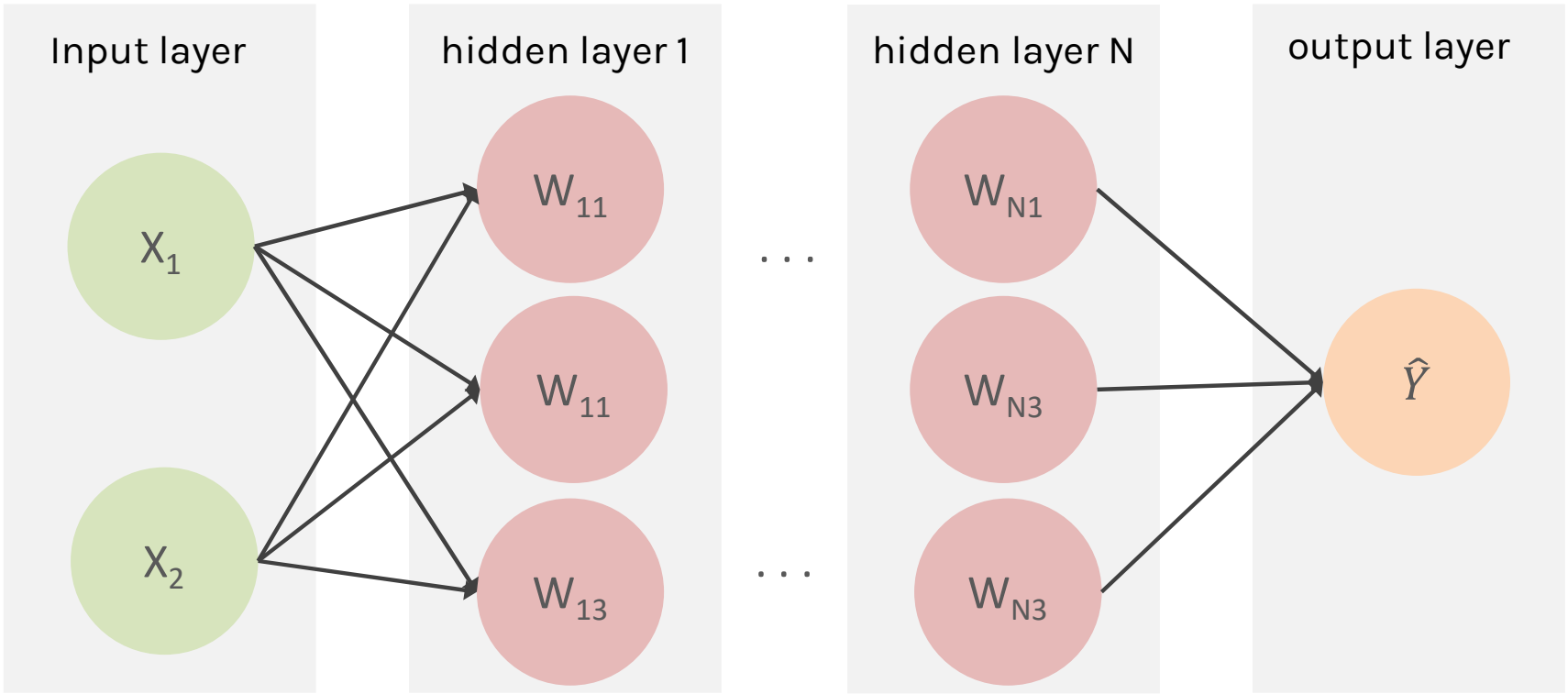
Adding another hidden layer

Anatomy of artificial neural network (ANN)



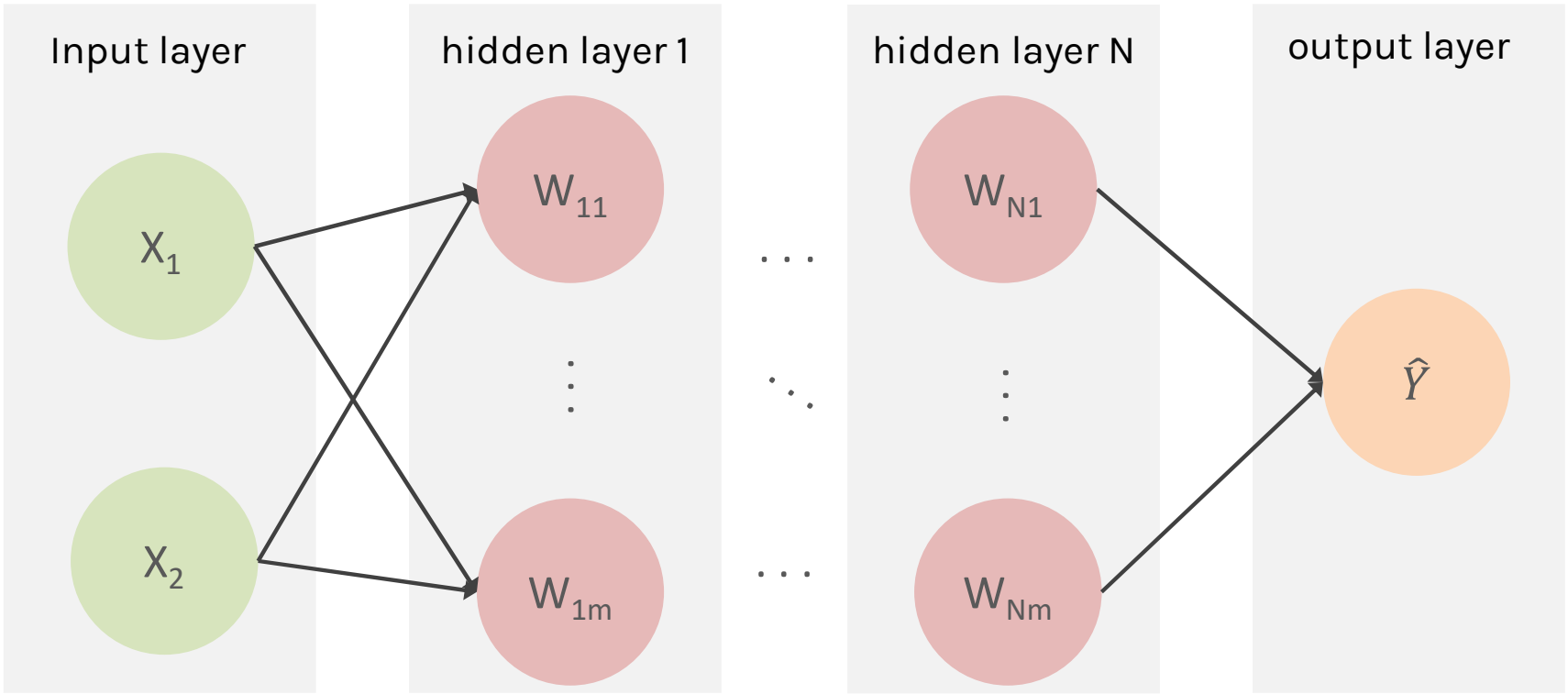
ANN with N hidden layers

Anatomy of artificial neural network (ANN)



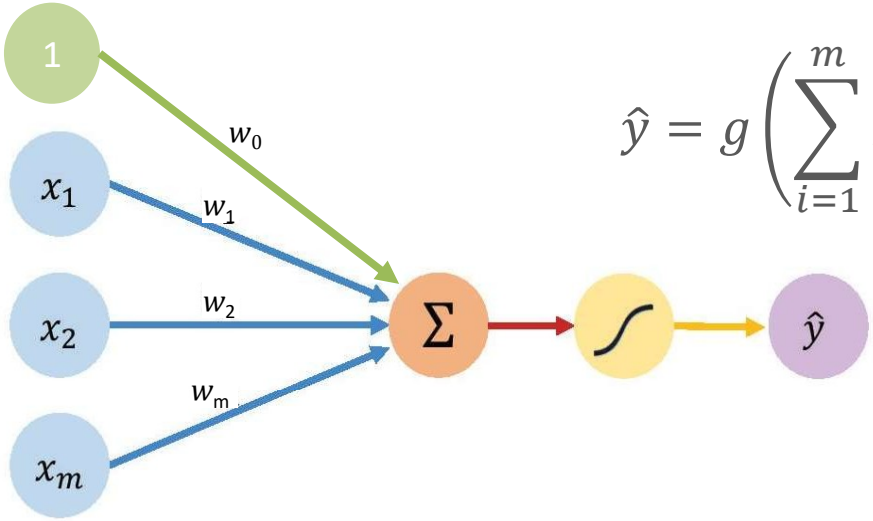
Adding anouther node in every hidden layer

Anatomy of artificial neural network (ANN)



Number of nodes in each hidden layer could be provisional

The Perceptron: Forward Propagation



$$\hat{y} = g\left(\sum_{i=1}^{m} x_i \cdot w_i\right)$$

Output with bias

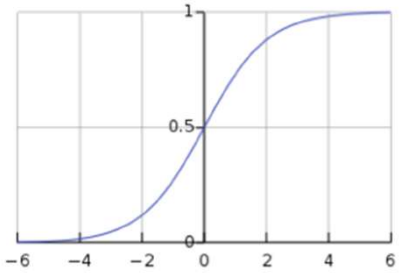
$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i \cdot w_i\right)$$

Output in matrix form

$$\hat{y} = g(w_0 + \boldsymbol{X}^T \boldsymbol{W})$$

$$\boldsymbol{X} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_m \end{bmatrix} \quad \boldsymbol{W} = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_m \end{bmatrix}$$

Inputs Weights Sum Non-Linearity Output

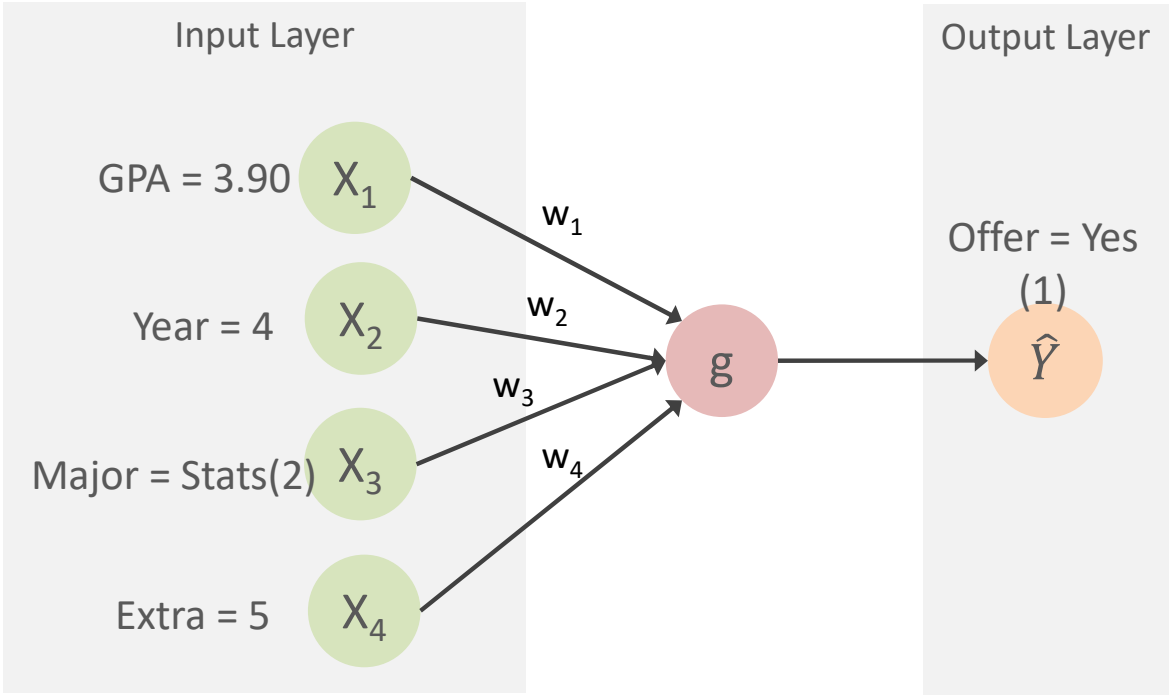


sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The Perceptron: Example

- How to predict if an internship applicant should be offered an interview based on their GPA, year in school, major, and number of extracurricular activities?



$$Probability\ of\ offer = \frac{1}{1 - e^{-X}}$$

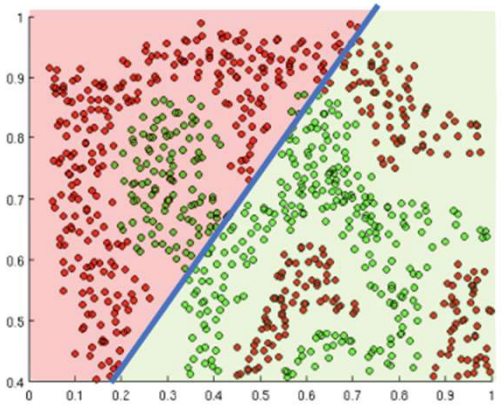
$$X = w_1 GPA + w_2 Year + w_3 Major + w_4 Extra + b$$

The algorithm should "learn" the best values for the weights and the bias

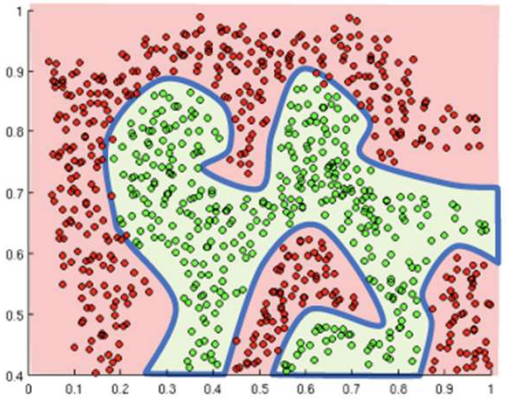
Activation function

$$h = f(W^T X + b)$$

- The activation function should:
  - Provide non-linearity
  - Ensure gradients remain large through hidden unit
- Common choices are:
  - sigmoid
  - Tanh
  - ReLU, leaky ReLU, Generalized ReLU, MaxOut
  - Softplus
  - swish



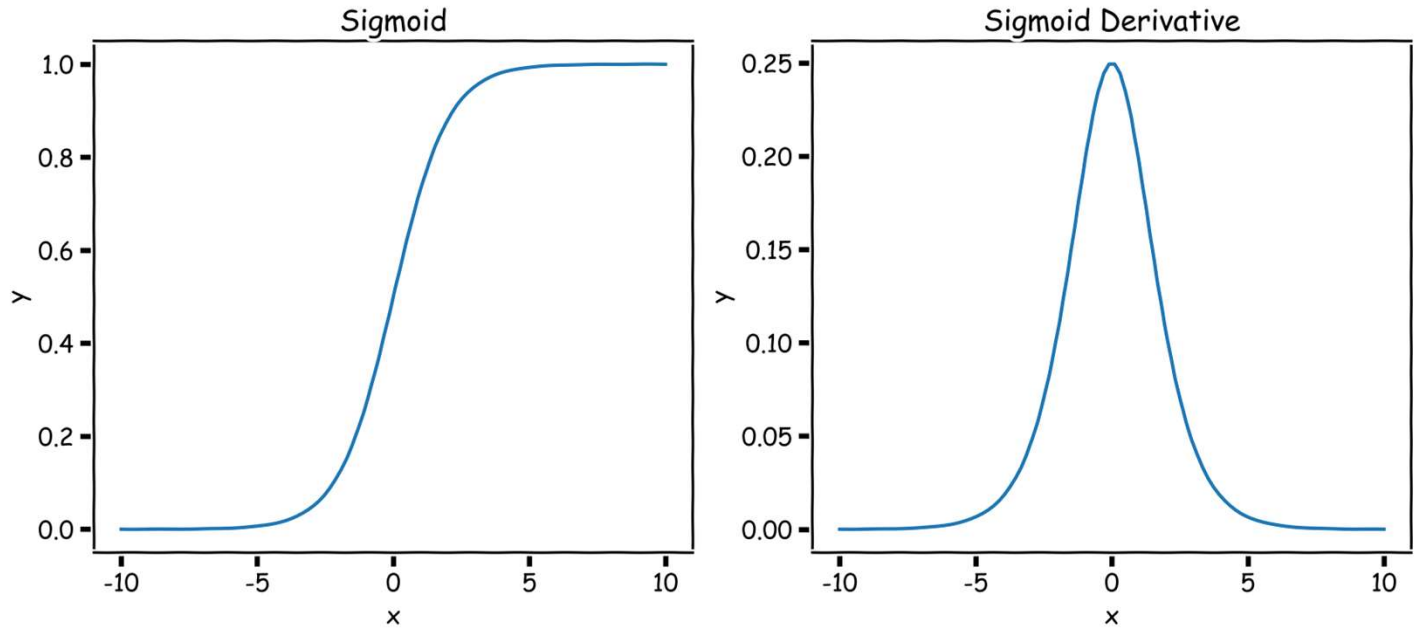
Linear Activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Sigmoid (aka Logistic)

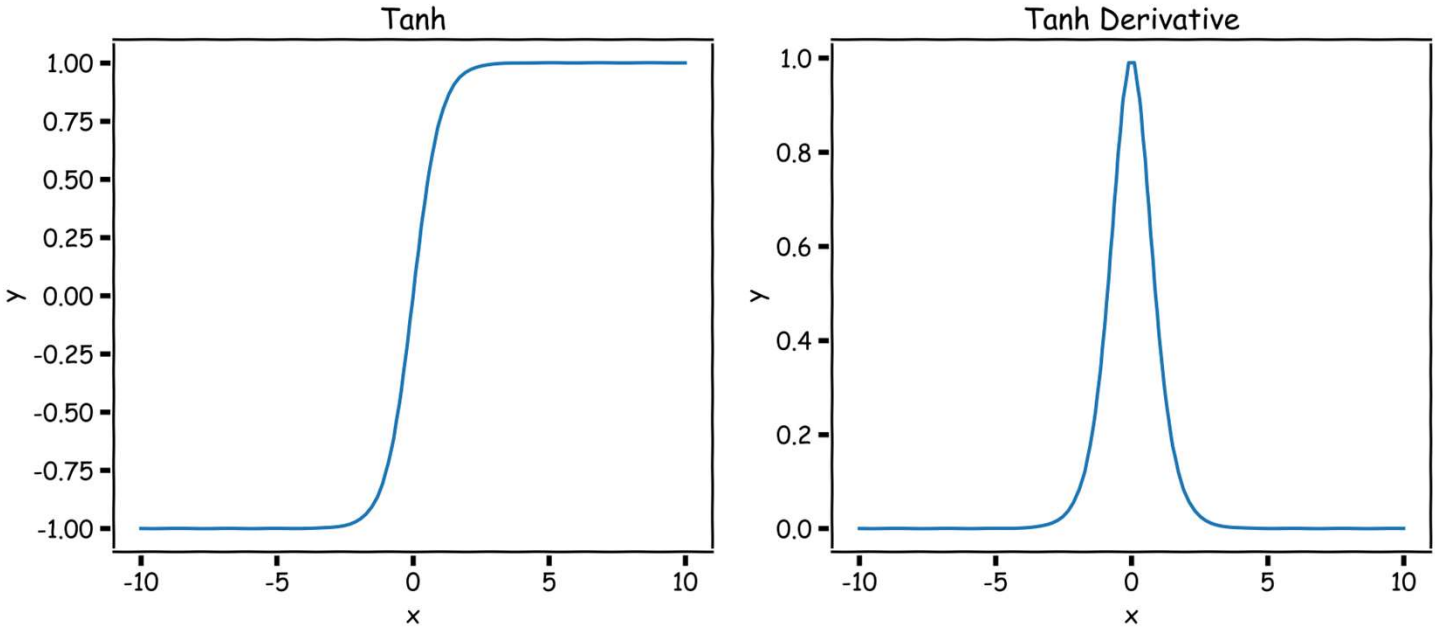
$$y = \frac{1}{1 + e^{-x}}$$



Derivative is **zero** for much of the domain.
This leads to “vanishing gradients” in backpropagation.

Hyperbolic Tangent (Tanh)

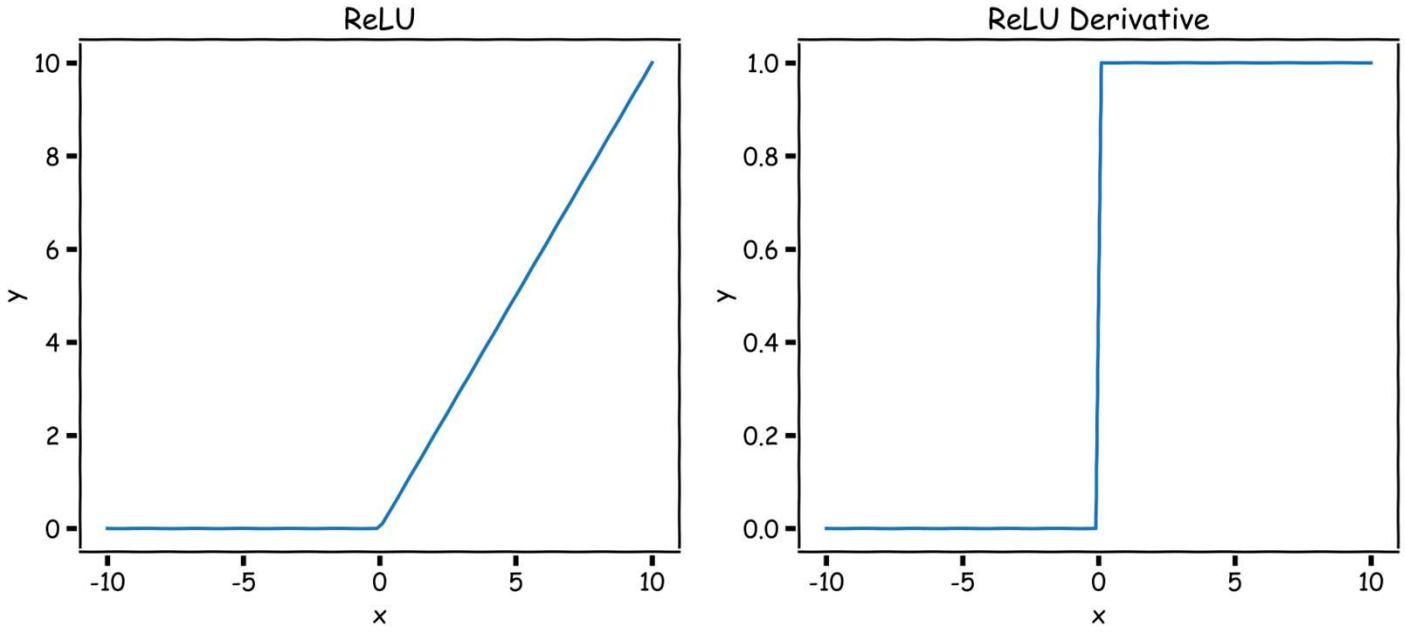
$$y = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



Same problem of “vanishing gradients” as sigmoid. .

Rectified Linear Unit (ReLU)

$$y = \max(0, x)$$



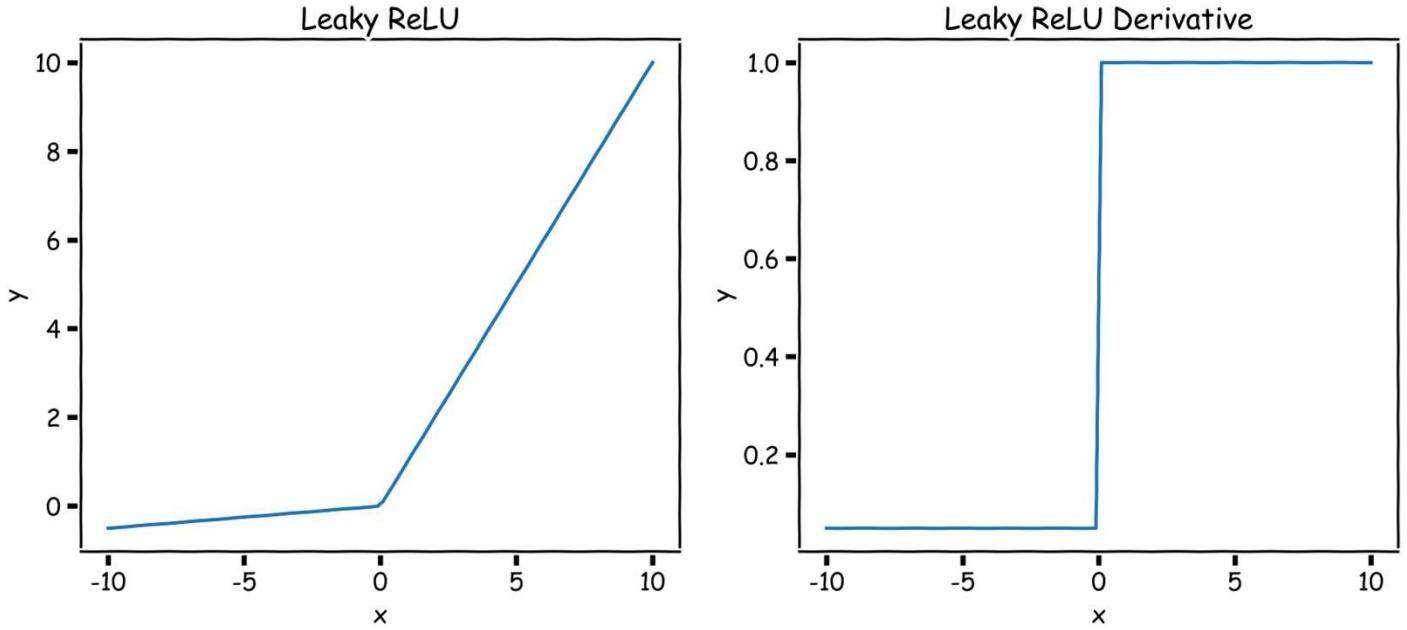
Two major advantages:

1. No vanishing gradient when x > 0
2. Provides sparsity (regularization) since y = 0 when x < 0

Leaky ReLU

$$y = \max(0, x) + \alpha \min(0, x)$$

where α takes a small value



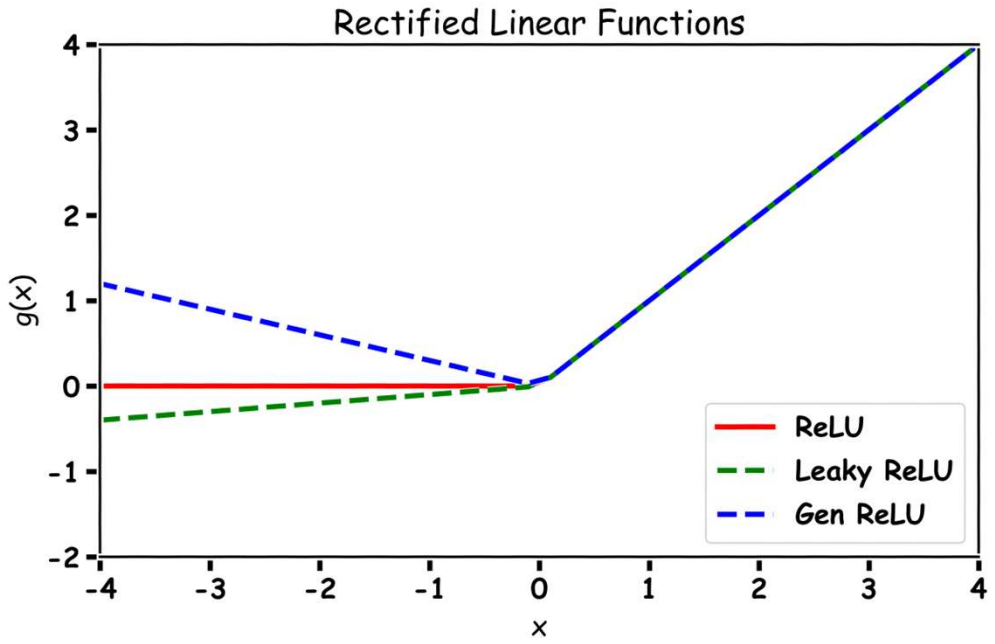
Tries to fix “dying ReLU” problem: derivative is non-zero everywhere.

Some people report success with this form of activation function, but the results are not always consistent

Generalized ReLU

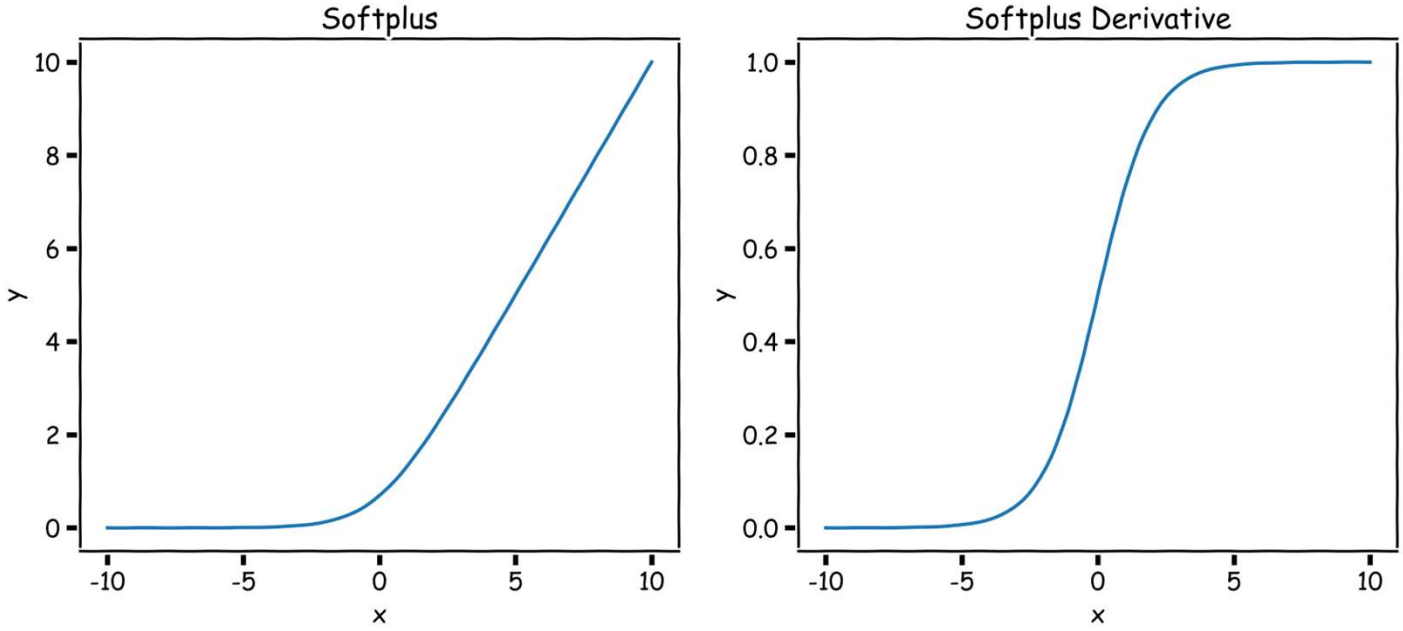
$$g(x_i, \alpha) = \max\{a, x_i\} + \alpha \min\{0, x_i\}$$

Generalization: for α_i>0



softplus

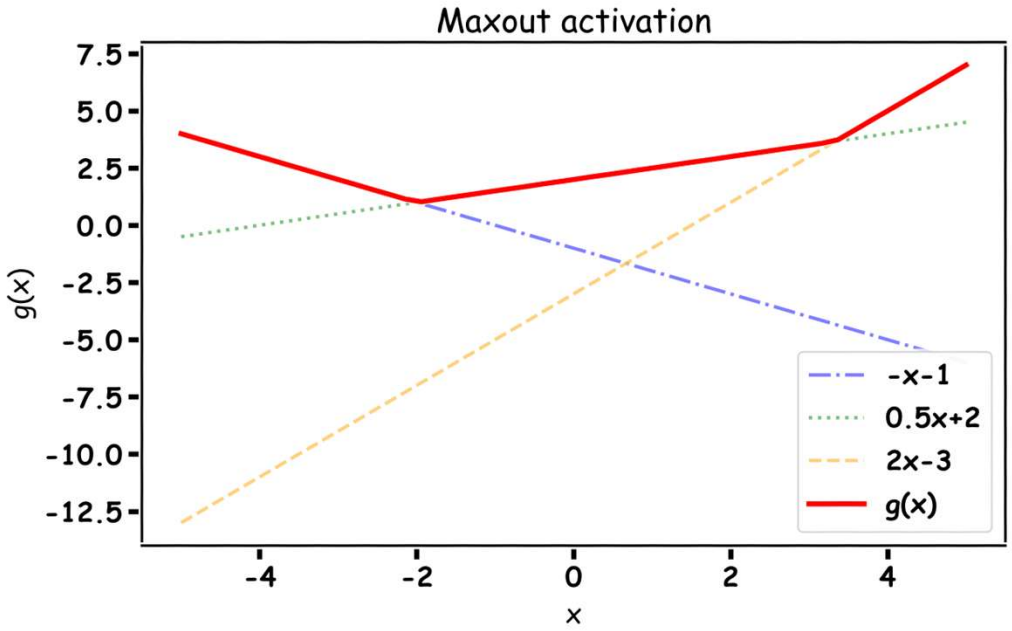
$$y = \log(1 + e^x)$$



The logistic sigmoid function is a smooth approximation of the derivative of the rectifier

Maxout

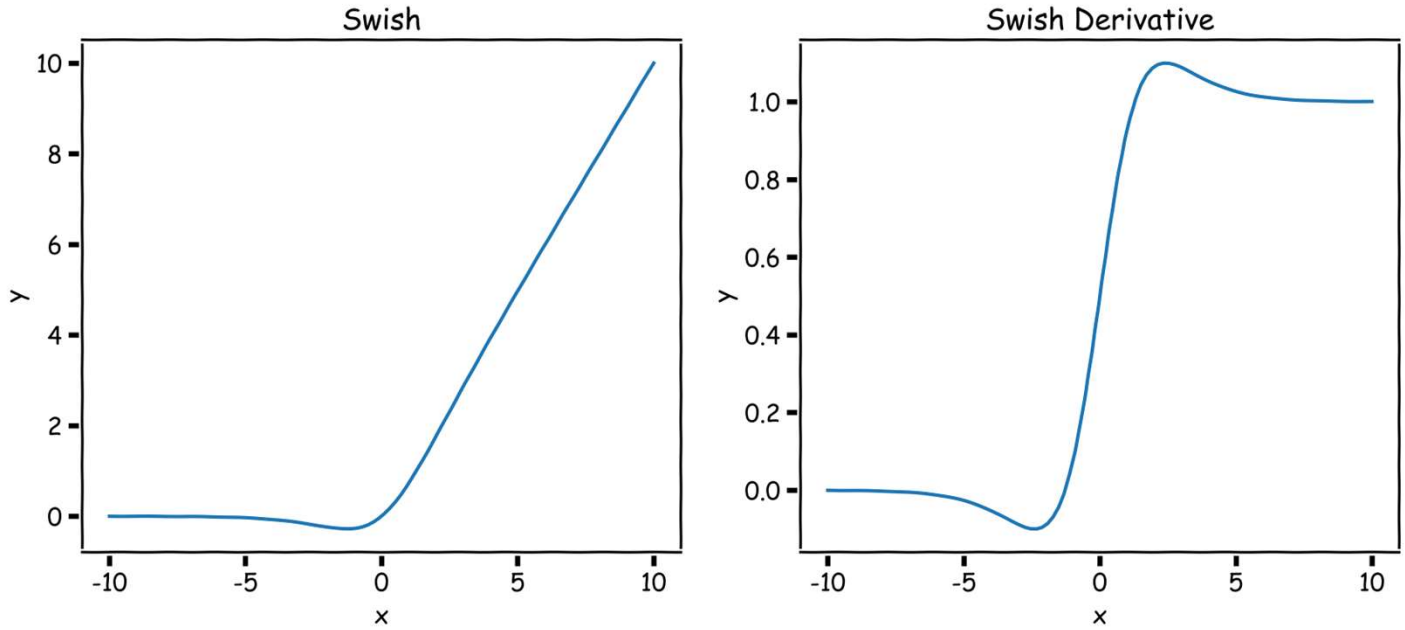
$$g(x) = \max_{i \in \{1,\ldots,k\}} \alpha_i x_i + \beta$$



Max of k linear functions. Directly learn the activation function.

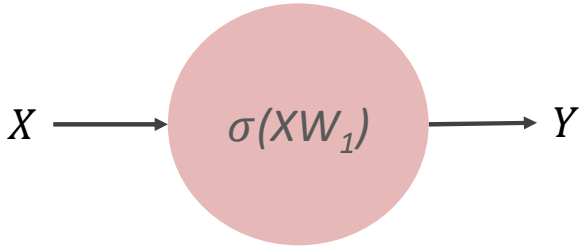
Swish: A Self-Gated Activation Function

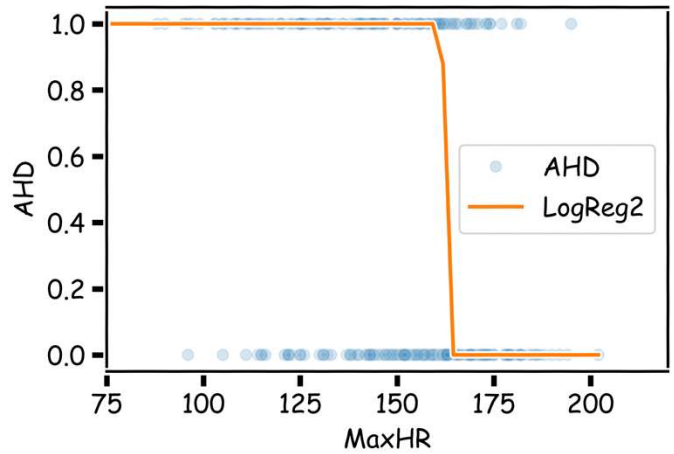
$$g(x) = x\,\sigma(x) = x/(1 + e^{-x})$$

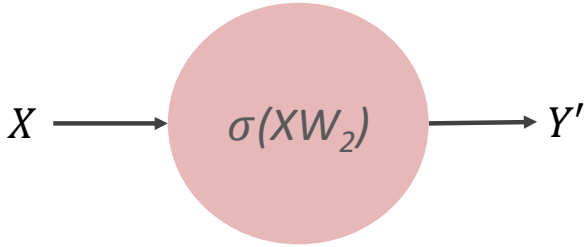


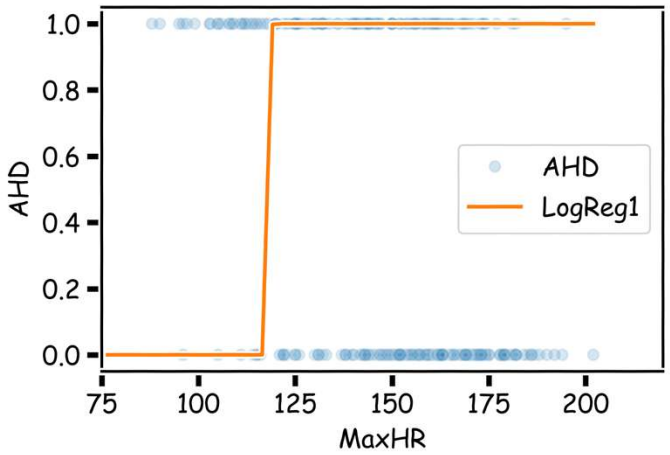
Currently, the most successful and widely-used activation function is the ReLU. Swish tends to work better than ReLU on deeper models across several challenging datasets.

Combining neurons allows us to model interesting functions

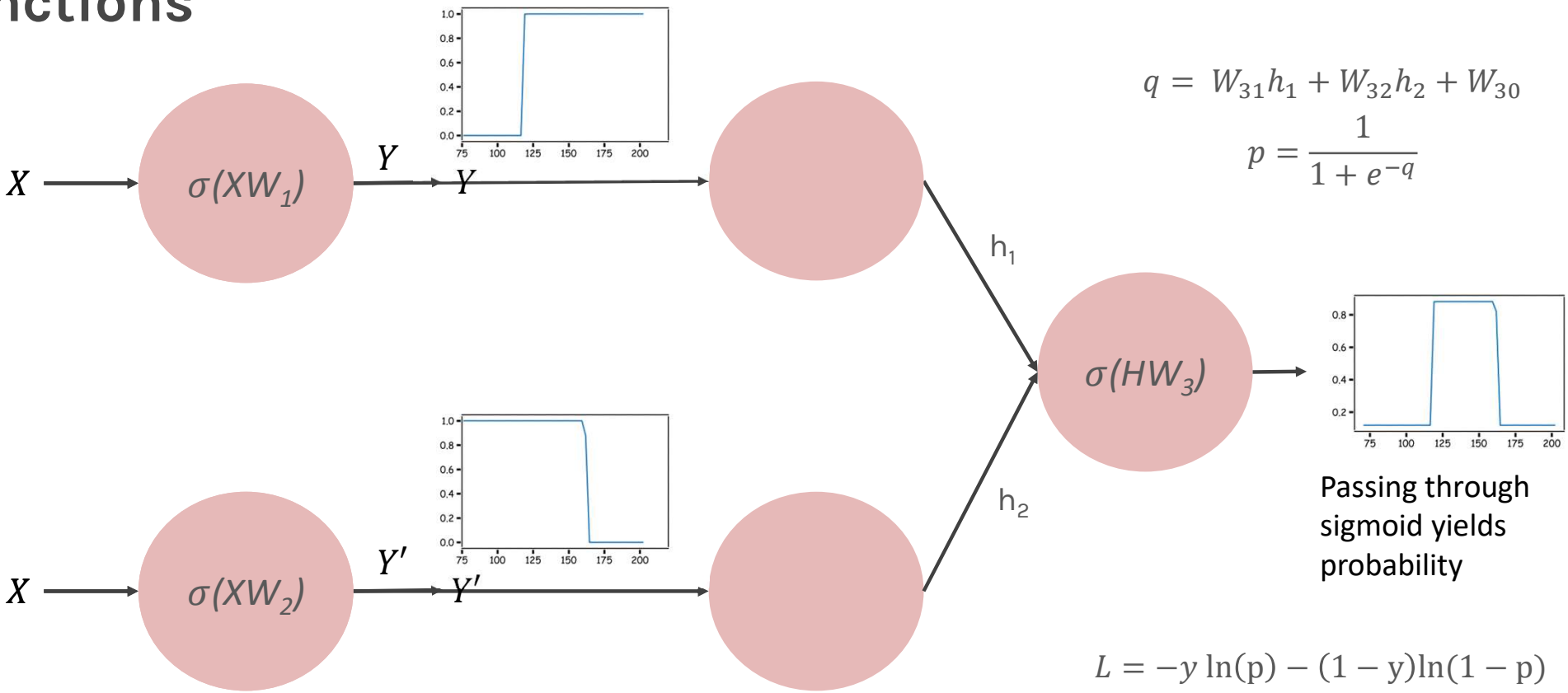








Combining neurons allows us to model interesting functions



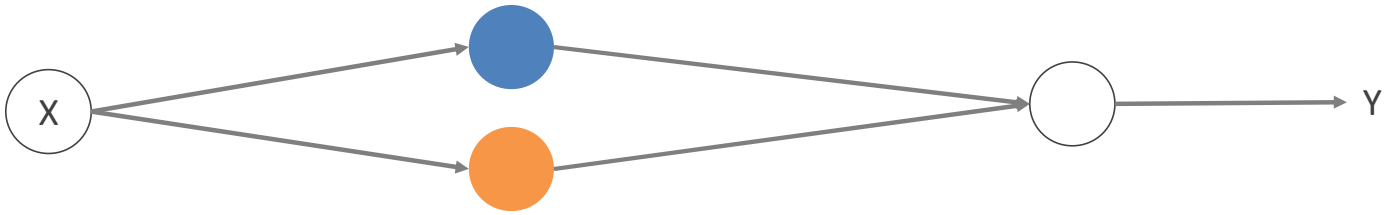
$$q = W_{31}h_1 + W_{32}h_2 + W_{30}$$

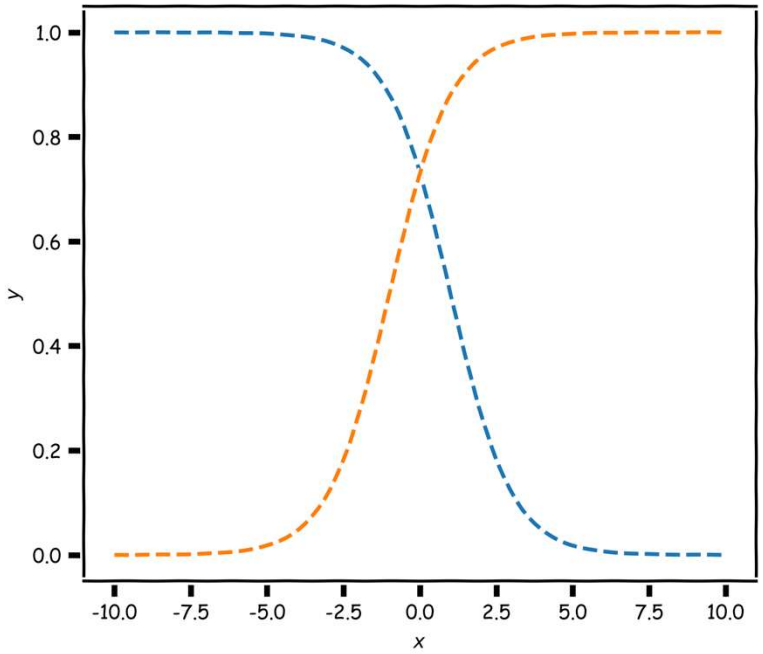
$$p = \frac{1}{1 + e^{-q}}$$

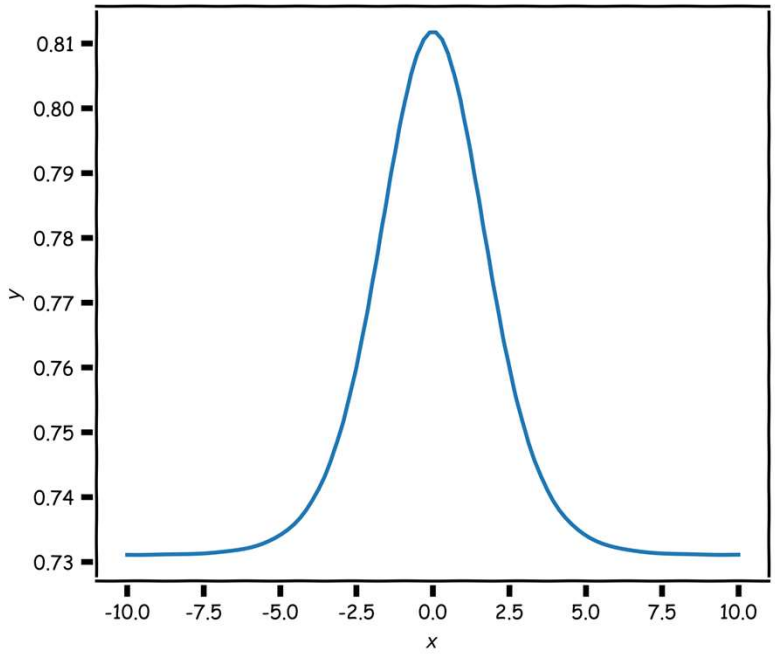
Passing through sigmoid yields probability

$$L = -y\ln(\mathrm{p}) - (1 - \mathrm{y})\ln(1 - \mathrm{p})$$

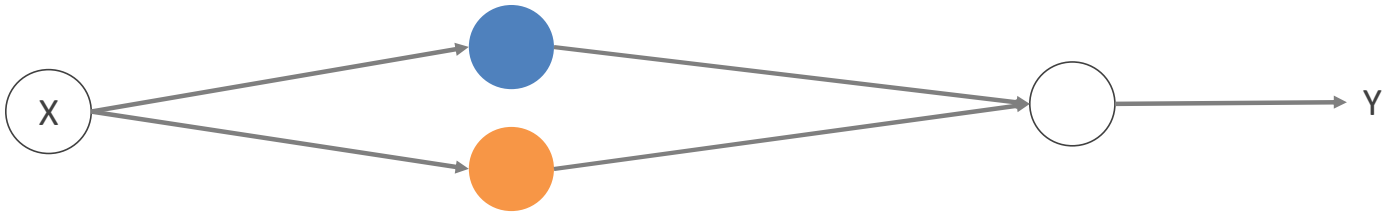
Combining neurons allows us to model interesting functions

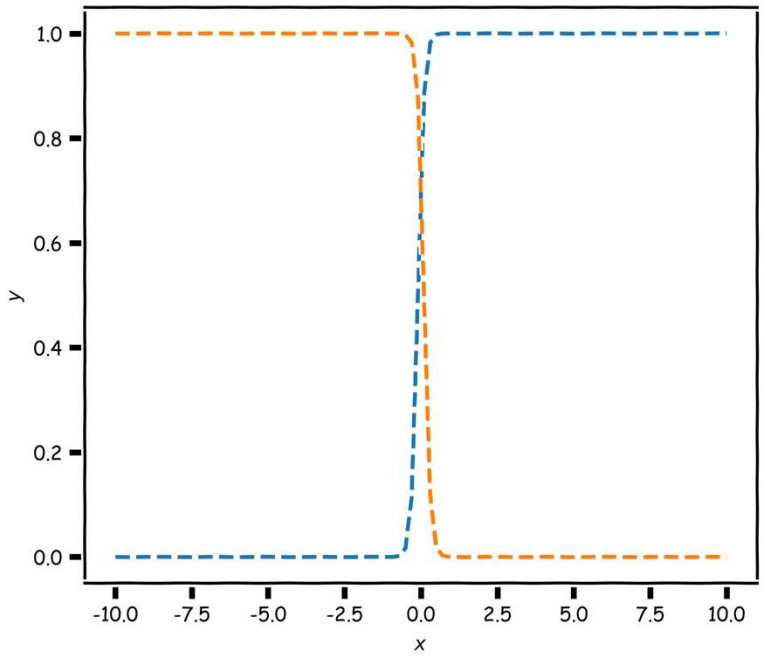


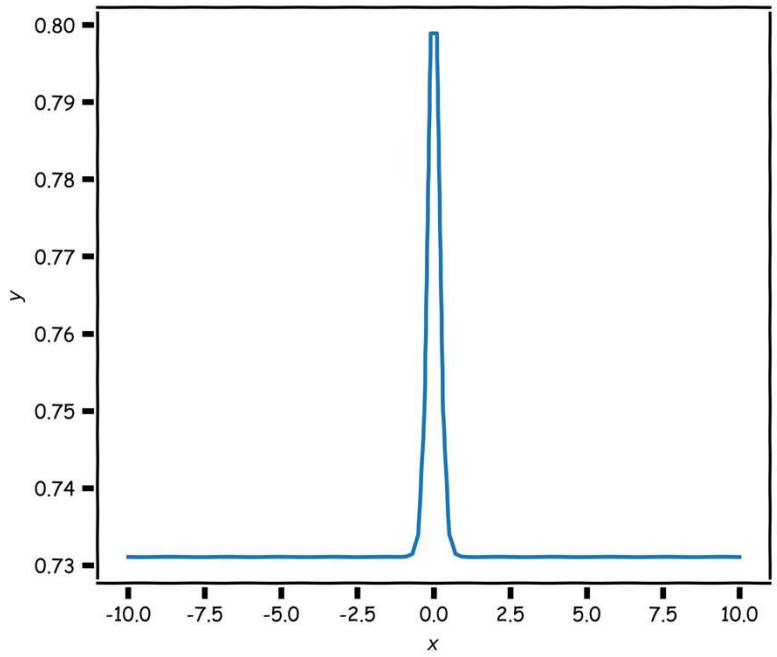




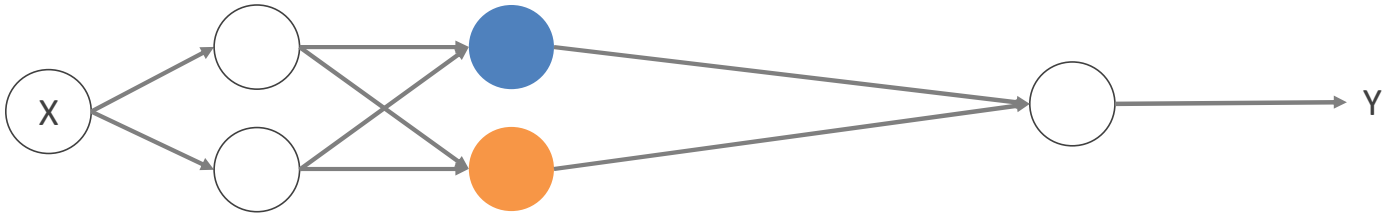
Combining neurons allows us to model interesting functions

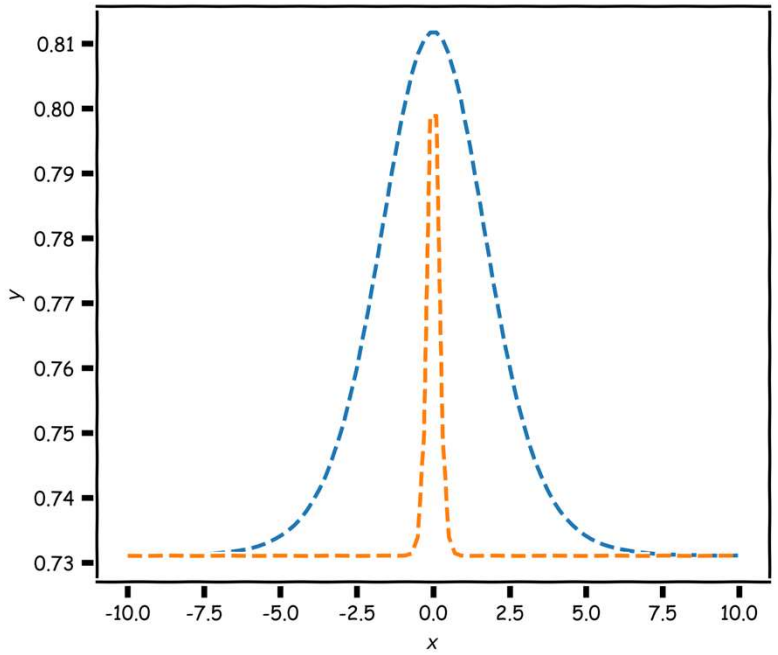


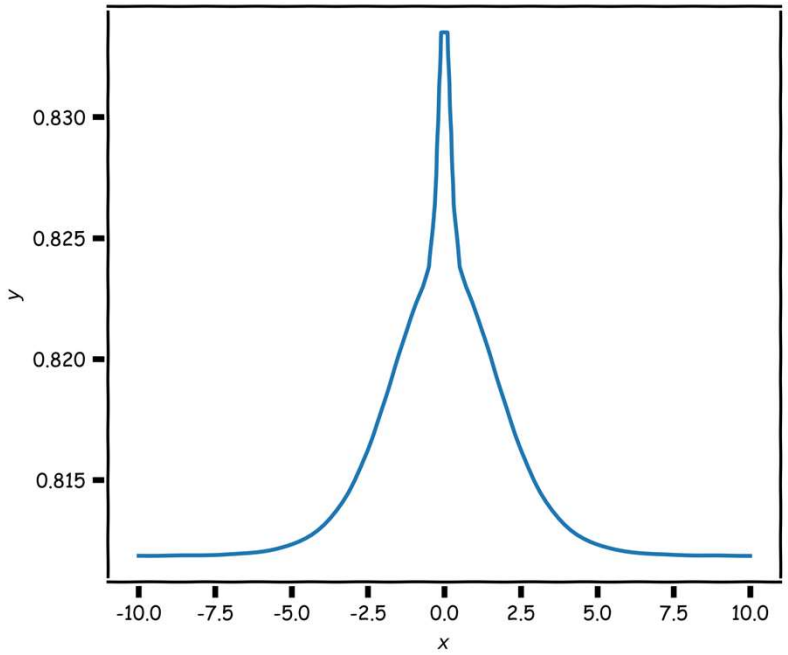




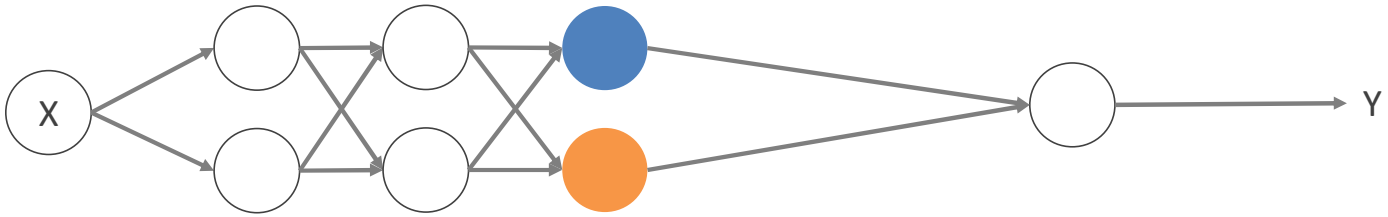
Combining neurons allows us to model interesting functions

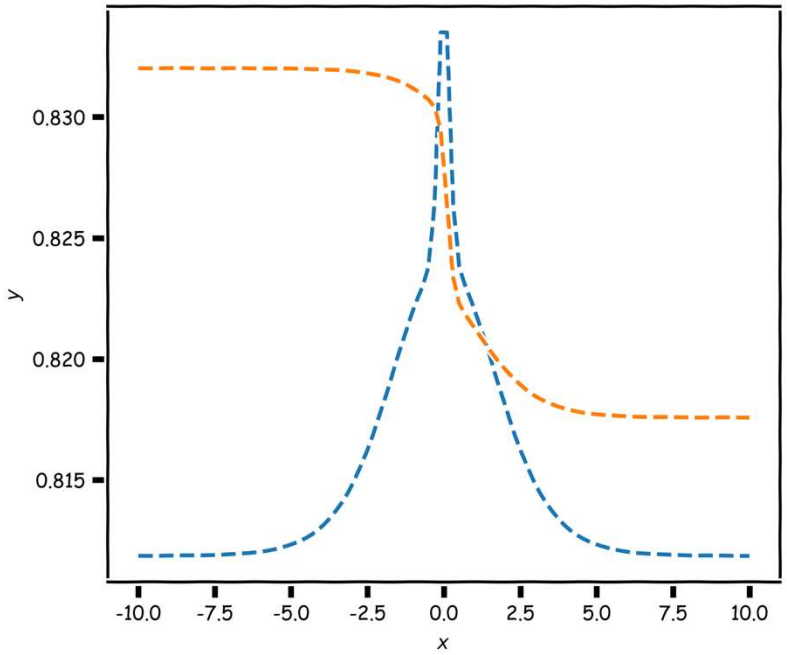


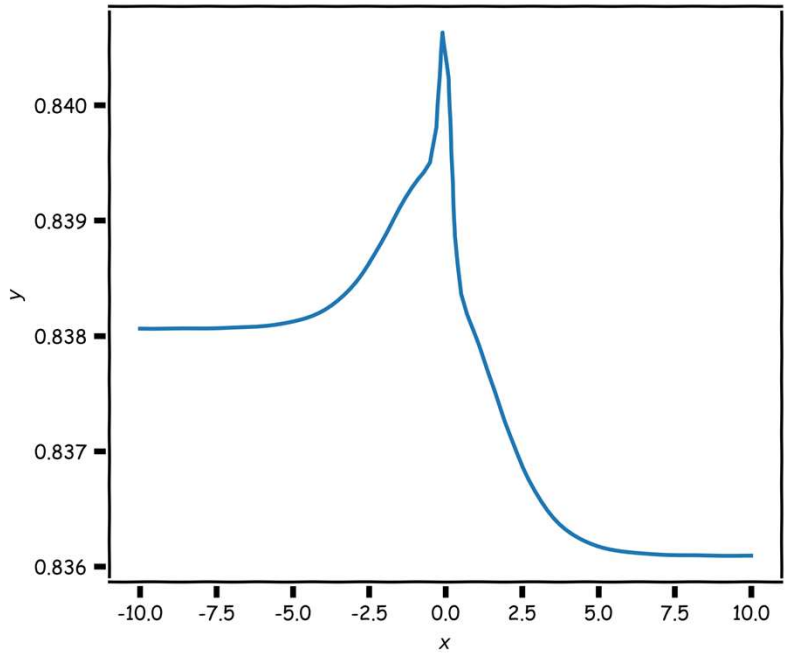




Combining neurons allows us to model interesting functions







Loss Function

- Likelihood for a given point:

$$p(y_i|W;x_i)$$

- Assume independency, likelihood for all measurements:

$$L(W;X,Y) = p(Y|W;X) = \prod_i p(y_i|W;x_i)$$

- Maximize the likelihood, or equivalently maximize the log-likelihood:

$$\log L(W;X,Y) = \sum_i \log p(y_i|W;x_i)$$

- Turn this into a loss function:

$$\mathcal{L}(W;X,Y) = -\log L(W;X,Y)$$

Loss Function: Takes all these results and averages them and tells us how bad or good the computer or those weights are.

Loss Function

- Do not need to design separate loss functions if we follow this simple procedure

- Distribution is **Normal** then likelihood is:

$$p(y_i|W;x_i) = \frac{1}{\sqrt{\{2\pi^2\sigma\}}} e^{-\frac{(y_i-\hat{y}_i)^2}{2\sigma\hat{}2}}$$

MSE

Mean Squared Error

$$\mathcal{L}(W;X,Y) = \sum_i (y_i - \hat{y}_i)^2$$

- Distribution is **Bernouli** then likelihood is:

$$p(y_i|W;x_i) = p_i^{y_i}(1-p_i)^{1-y_i}$$

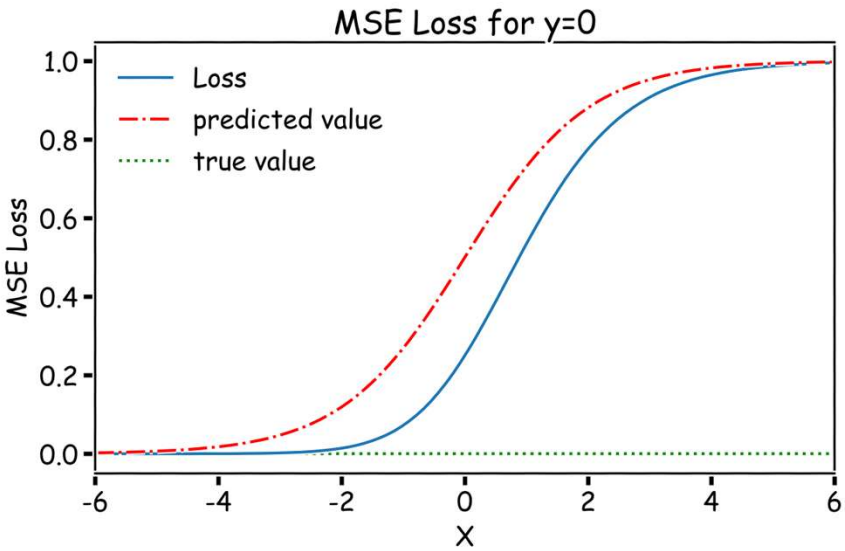
Cross-Entropy

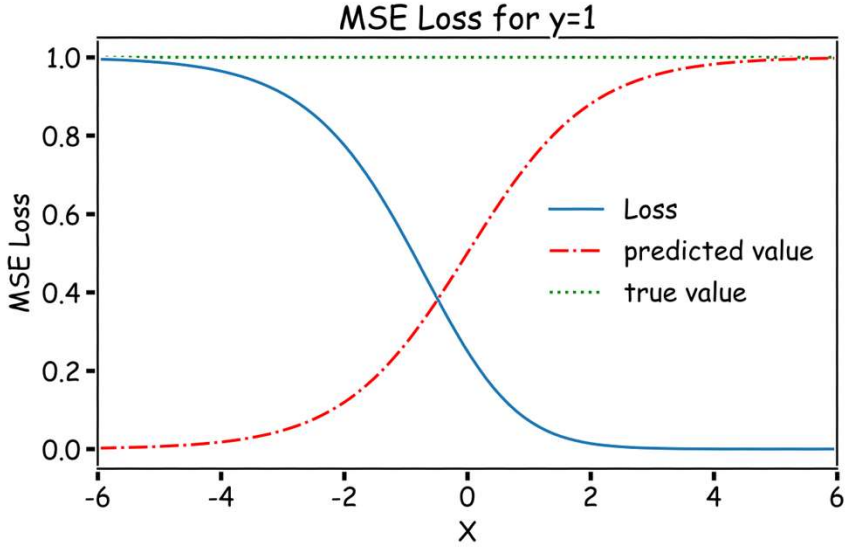
$$\mathcal{L}(W;X,Y) = -\sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)]$$

Loss Function

- Example: sigmoid output + squared loss

$$L_{sq} = (y - \hat{y})^2 = \big(\mathrm{y} - \sigma(x)\big)^2$$

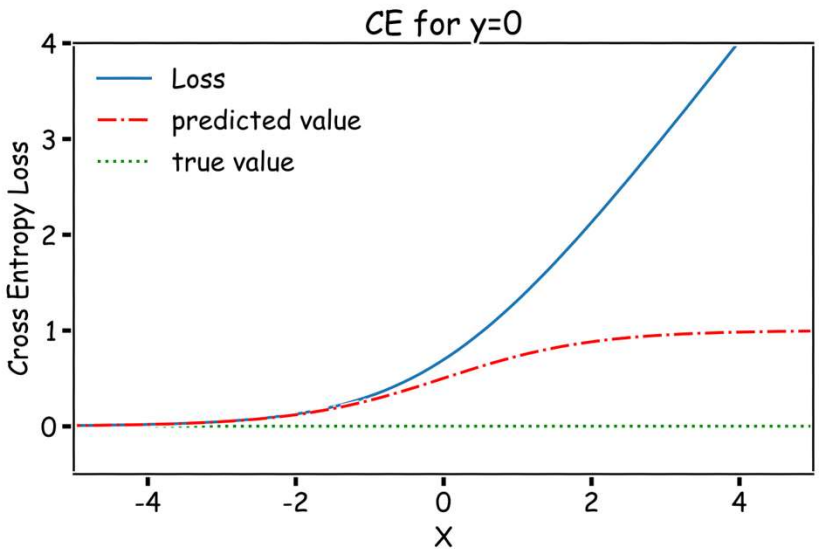


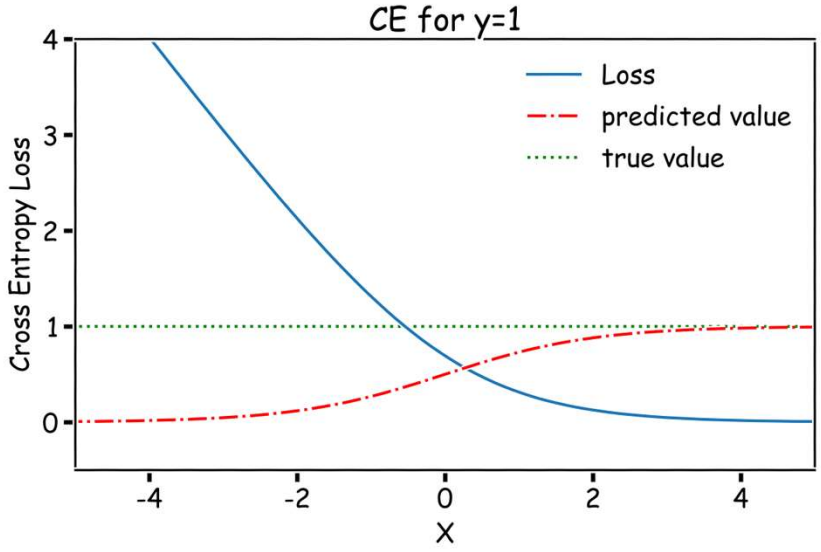


Loss Function

- Example: sigmoid output + cross-entropy loss

$$L_{ce}(y, \hat{y}) = -\{\, y \log \hat{y} + (1 - y) \log(1 - \hat{y})\}$$





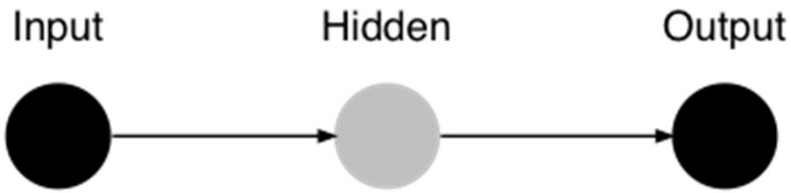
Saturates only when the model makes correct predictions

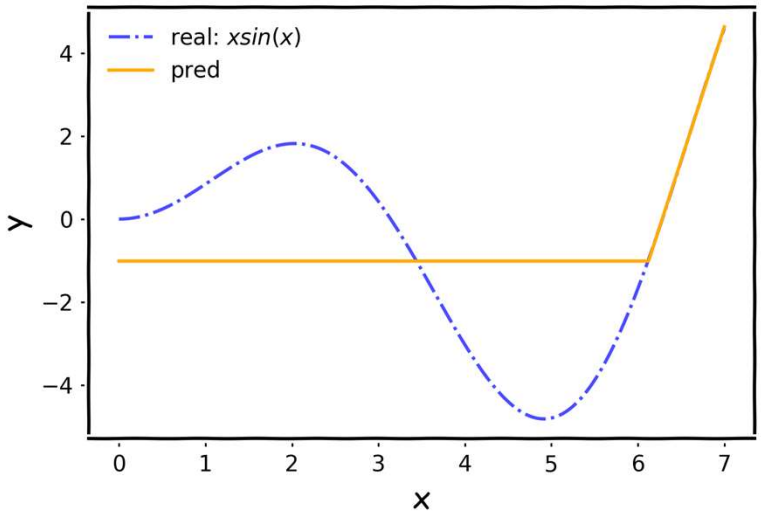
Output Units

Output Type	Output Distribution	Output layer	Cost Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinoulli	Softmax	Cross Entropy
Continuous	Gaussian	Linear	MSE (Mean Squared Error)
Continuous	Arbitrary	-	-

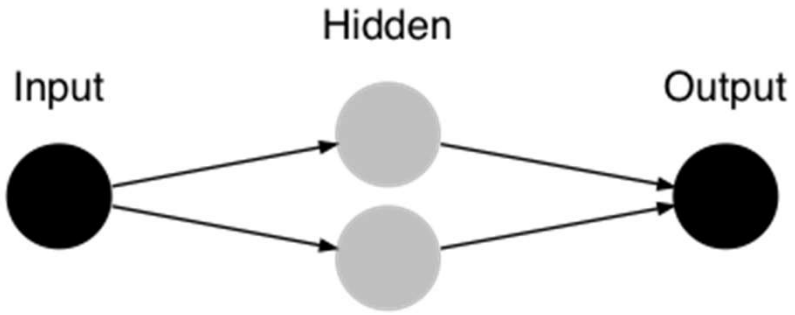
Multinoulli (Categorical): Generalization of Bernoulli to K classes

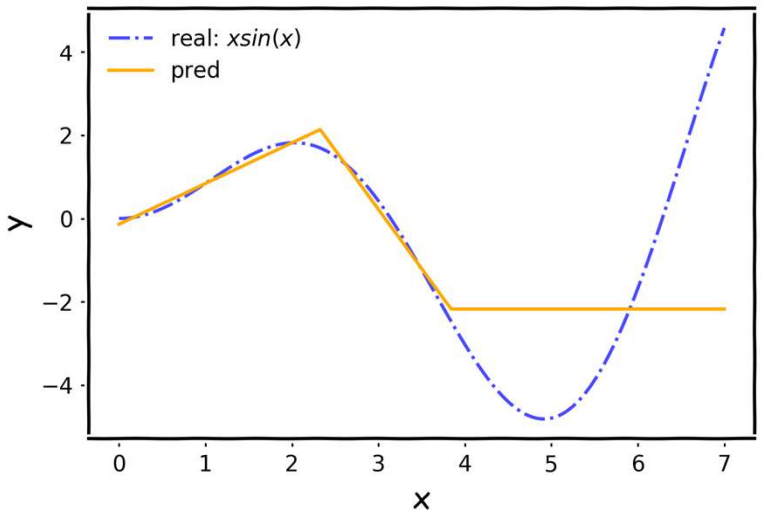
NN in action



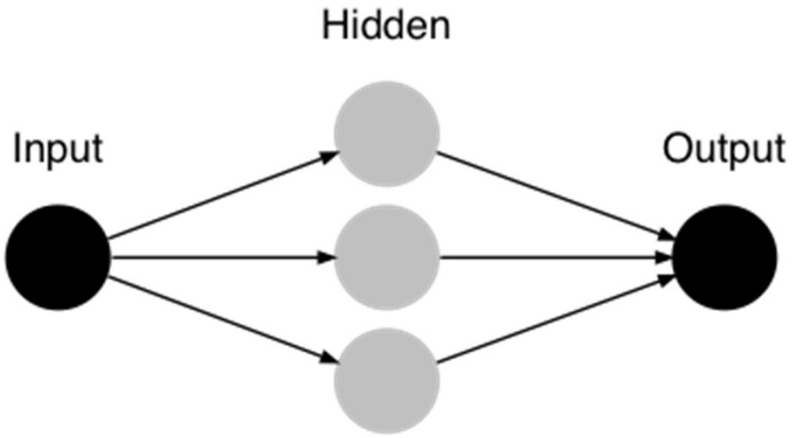


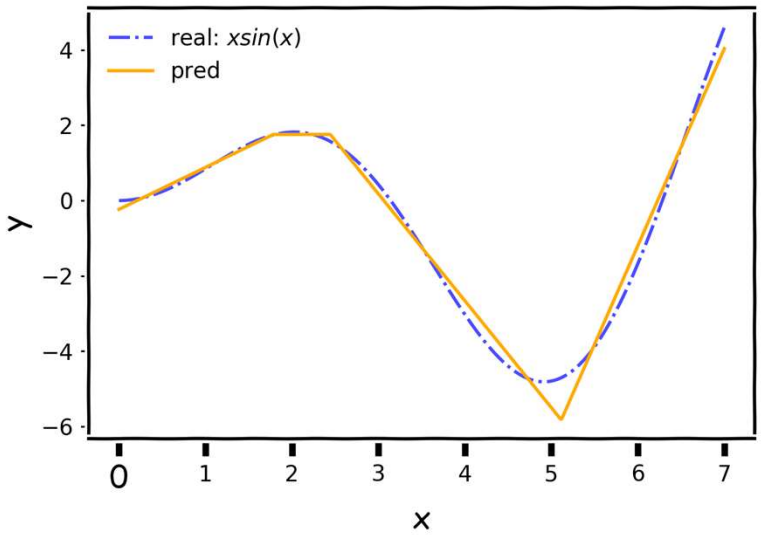
NN in action



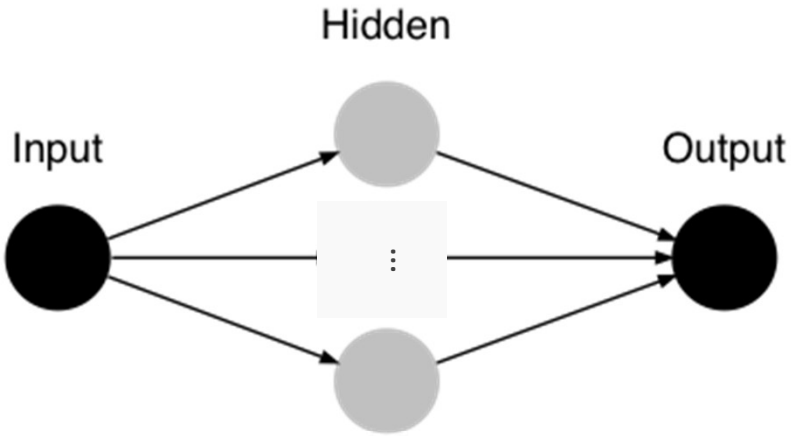


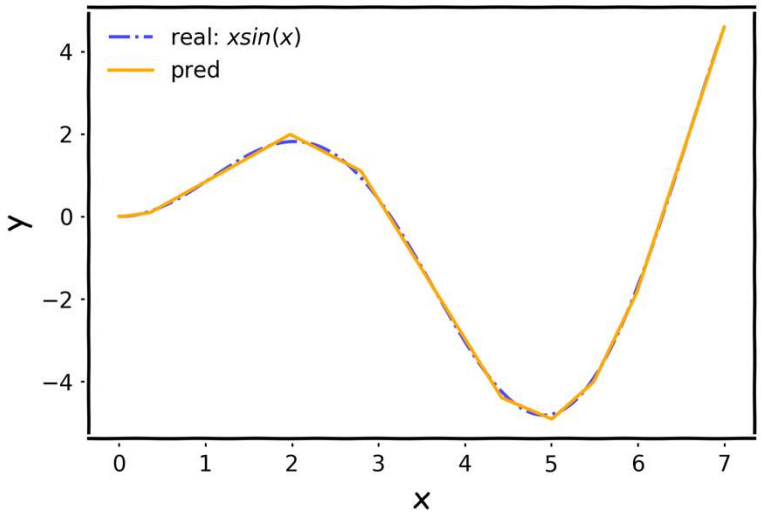
NN in action



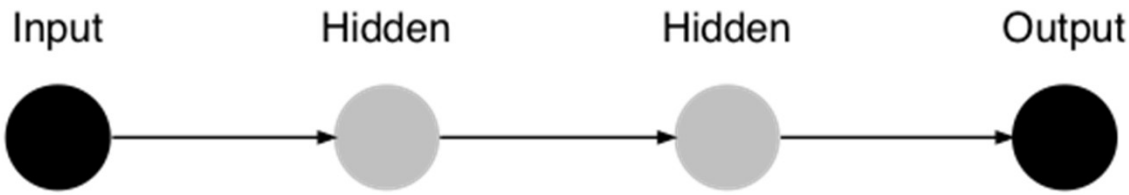


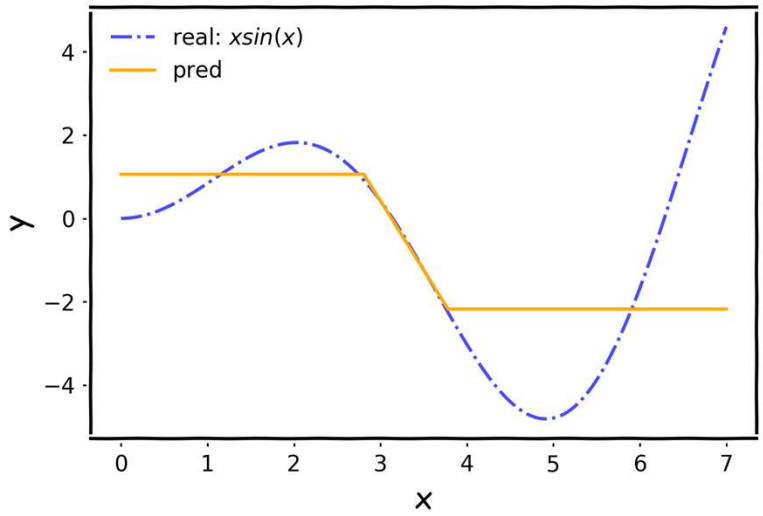
NN in action



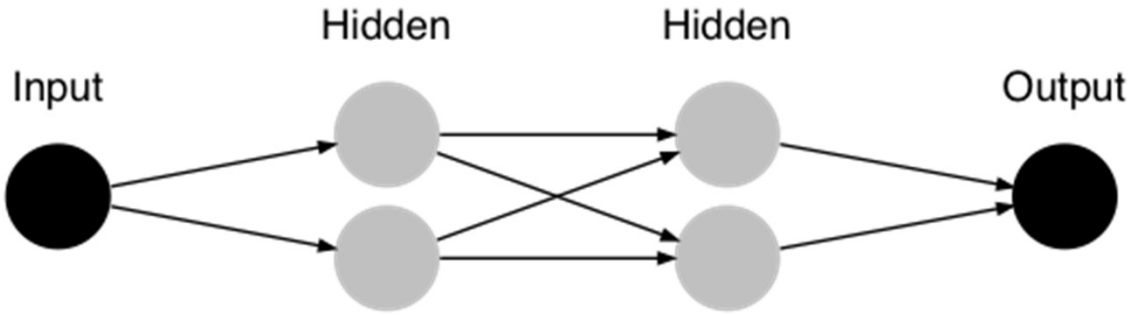


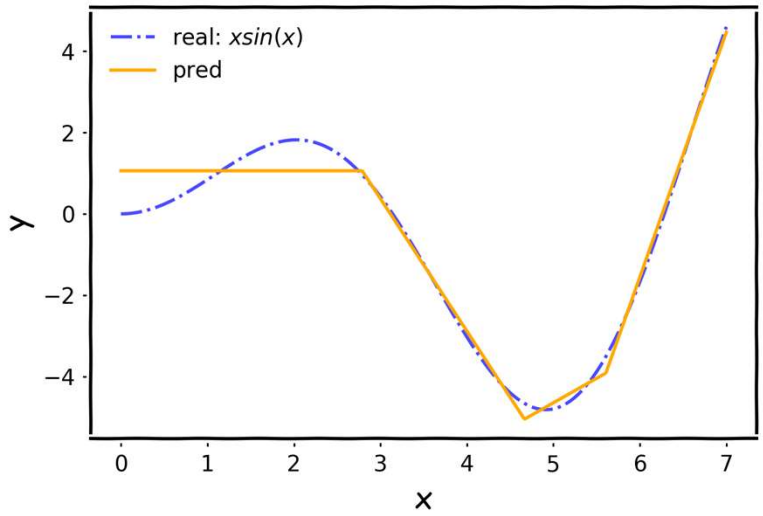
NN in action


Input
Hidden
Hidden
Output

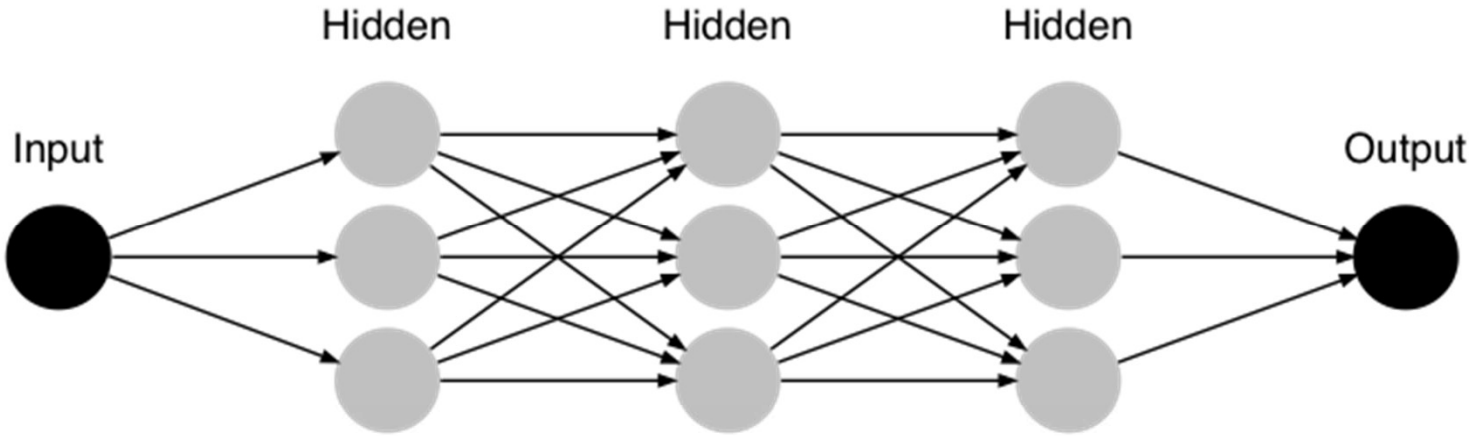

real: xsin(x)
pred
y
x
4
2
0
−2
−4
0
1
2
3
4
5
6
7

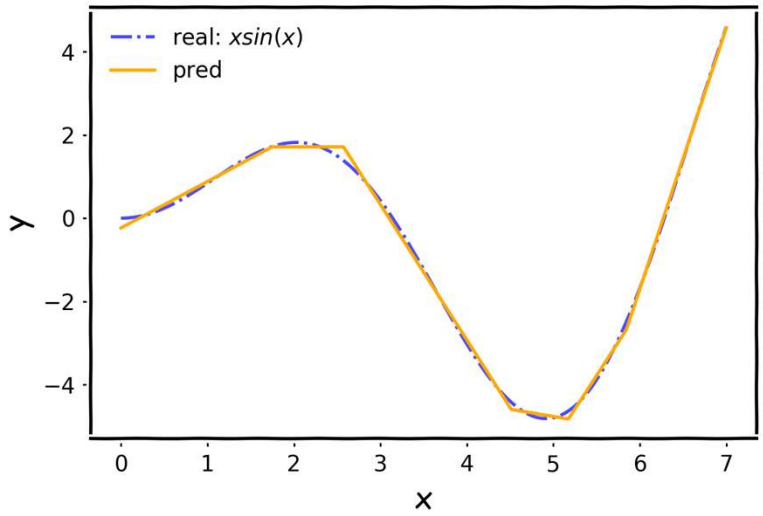
NN in action


Input
Hidden
Hidden
Output


real: xsin(x)
pred
y
x
4
2
0
−2
−4
0
1
2
3
4
5
6
7

NN in action


Hidden
Hidden
Hidden
Input
Output


real: xsin(x)
pred
y
x
4
2
0
−2
−4
0
1
2
3
4
5
6
7

Universal Approximation Theorem

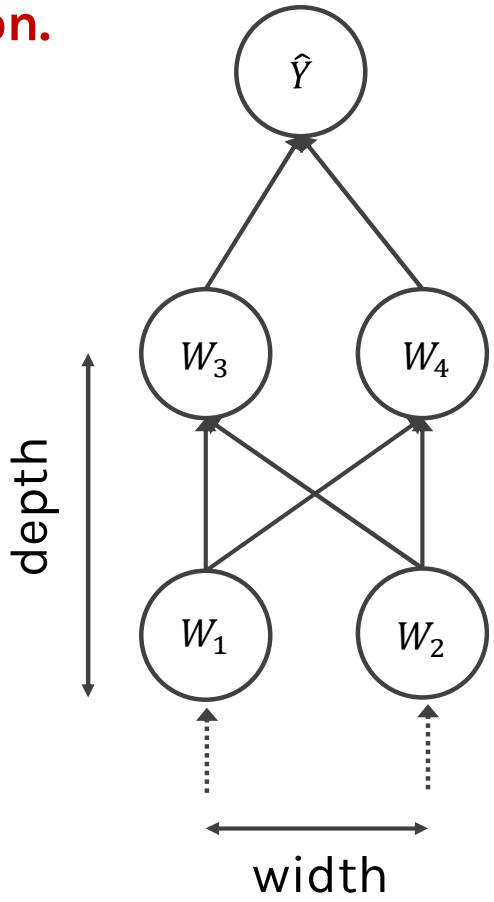
- **Think of a Neural Network as function approximation.**

  $Y = f(x) + \epsilon$

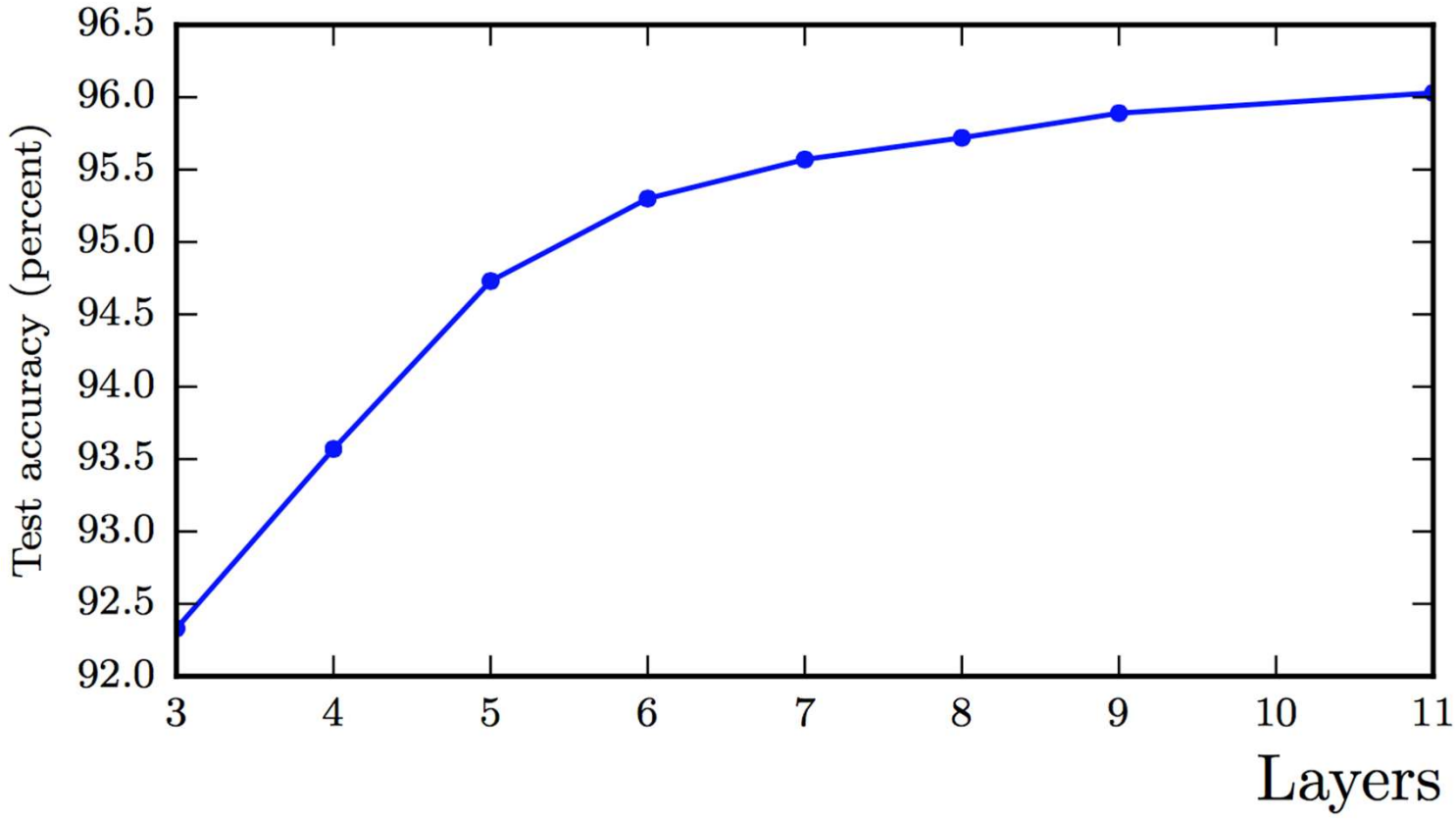
  $Y = \hat{f}(x) + \epsilon$

  NN: $\Longrightarrow \hat{f}(x)$

- One hidden layer is enough to *represent* an approximation of any function to an arbitrary degree of accuracy
- So why deeper?
  - Shallow net may need **(exponentially) more width**
  - Shallow net may overfit more



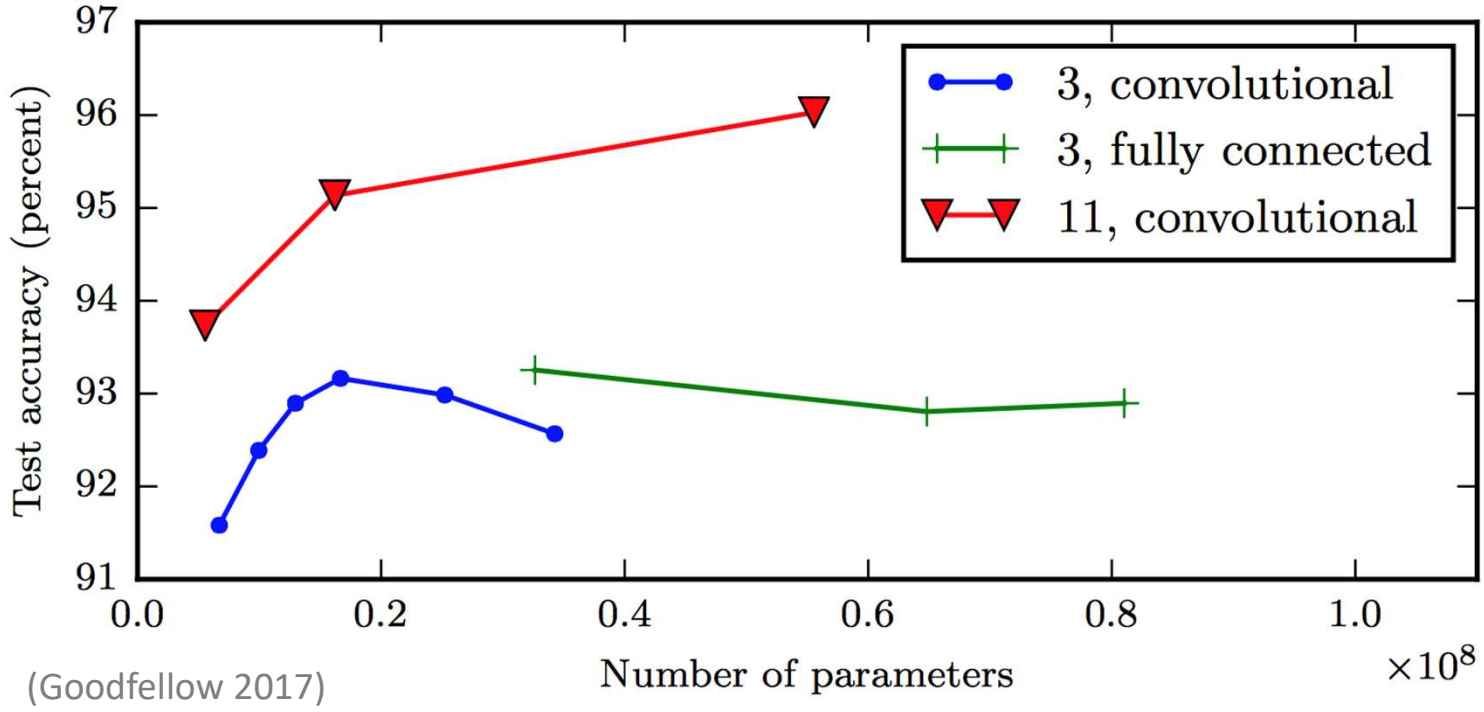
Better Generalization with Depth



(Goodfellow 2017)

Shallow Nets Overfit More

Depth helps, and it's not just because of more parameters



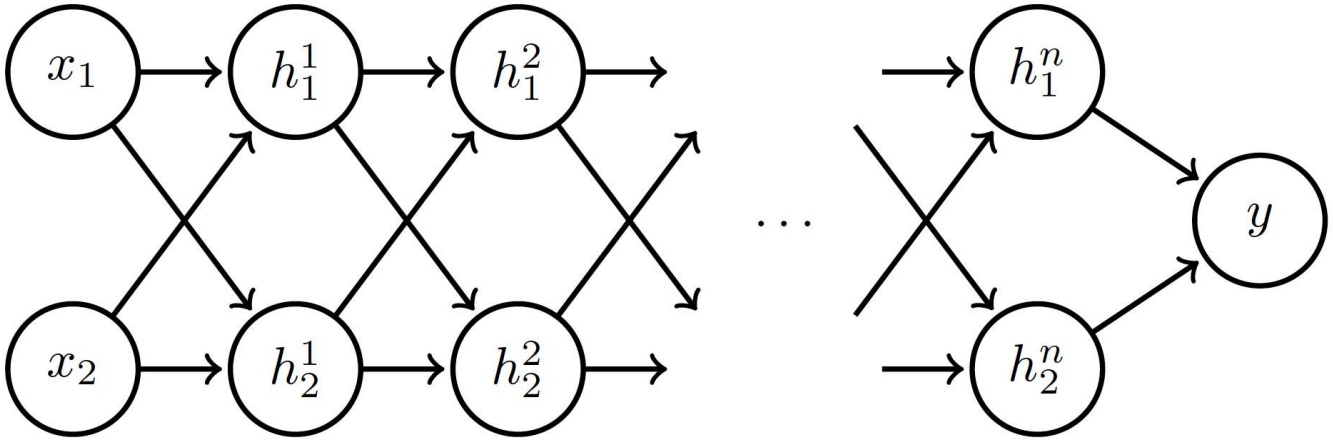
(Goodfellow 2017)

Don't worry about this word "convolutional". It's just a special type of neural network, often used for images.

The **11-layer net** generalizes better on the test set when controlling for number of parameters.

The 3-layer nets perform worse on the test set, even with similar number of total parameters.

The problem of Exploding and Vanishing Gradients



Linear activation:

$h^1 = Wx$

$h_i = Wh_{i-1}, \qquad i = 2, \ldots, n$

Suppose:

$$\boldsymbol{W} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}$$

$$\begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \qquad \cdots \qquad \begin{bmatrix} h_1^n \\ h_2^n \end{bmatrix} = \begin{bmatrix} a^n & 0 \\ 0 & b^n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Exploding and Vanishing Gradients

$$\begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \cdots \quad \begin{bmatrix} h_1^n \\ h_2^n \end{bmatrix} = \begin{bmatrix} a^n & 0 \\ 0 & b^n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Suppose:

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

CASE 1: a=1, b=2:

$$y \to 1, \qquad \nabla y \to \begin{bmatrix} n \\ n2^{n-1} \end{bmatrix}$$

Explodes!

CASE 2: a=0.5, b=0.9:

$$y \to 0, \qquad \nabla y \to \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

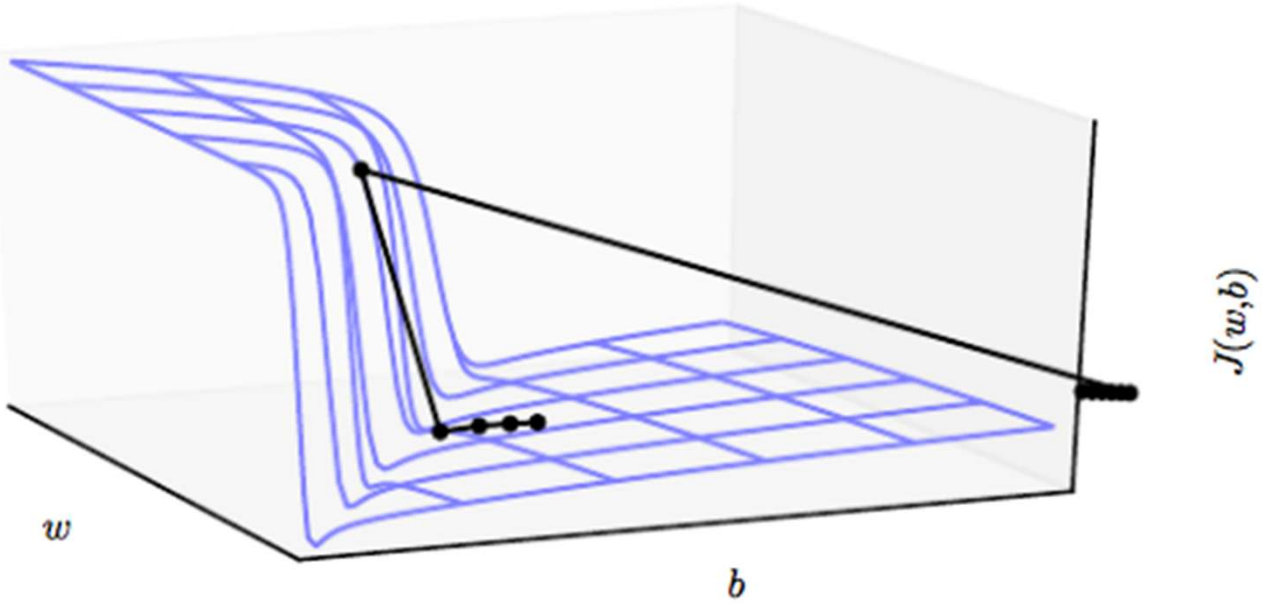
Vanishes!

Exploding and Vanishing Gradients

- Exploding gradients lead to cliffs
- Can be mitigated using **gradient clipping:** a technique that limits the magnitude of gradients during backpropagation to prevent them from becoming too large.
  - If the norm of the gradient exceeds a predefined threshold, it is scaled down to fit **within that limit.**

$$\text{if } \|g\| > u$$

$$g \leftarrow \frac{gu}{\|g\|}$$



Feature Normalization

- Features with large magnitudes or ranges can dominate smaller-scale features, leading to biased learning.
- Example: If one feature ranges between 0 and 1, and another ranges between 0 and 1000, the second feature will have a disproportionate impact on the model.
- Normalizing features ensures gradients remain well-scaled, improving optimization efficiency.
- In deep networks, unnormalized features can exacerbate issues like exploding or vanishing gradients, making training unstable.
- Enables Faster Convergence in Neural Networks

Feature Normalization

- Good practice to normalize features before applying learning algorithm:

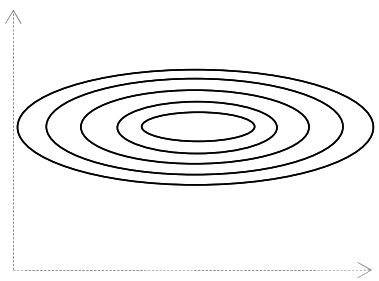
Feature vector

Vector of mean feature values

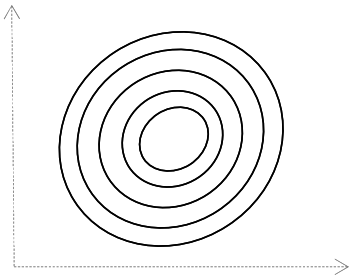
$$x' = \frac{x - \mu}{\sigma}$$

Vector of SD of feature values

- Features in **same scale**: mean 0 and variance 1
  - Speeds up learning



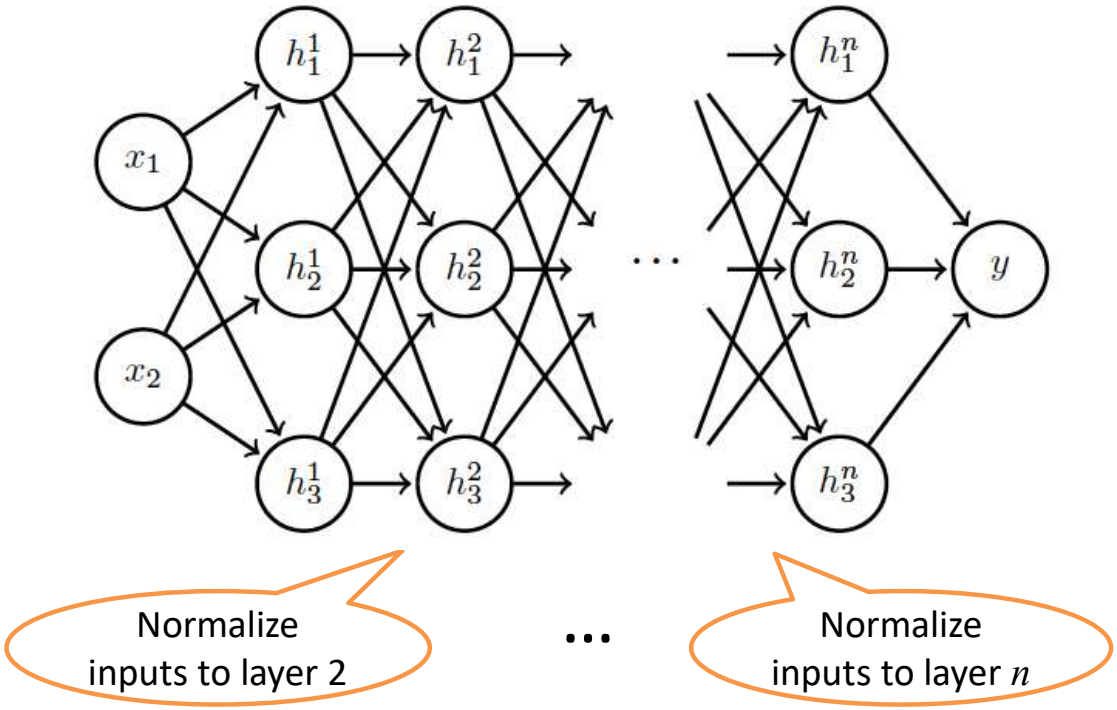
Before normalization



After normalization

Internal Covariance Shift

- The phenomenon where the distribution of activations (outputs of neurons) in each layer changes as the parameters of the previous layers are updated during training.
- This shift can slow down training and make optimization more challenging.



Batch Normalization

- Batch normalization normalizes the inputs to a layer for each mini-batch during training.
- It ensures that the activations of a layer have a consistent mean and variance, which helps stabilize the learning process.

$$H = \begin{bmatrix} H_{11} & \cdots & H_{1K} \\ \vdots & \ddots & \vdots \\ H_{N1} & \cdots & H_{NK} \end{bmatrix}$$

K hidden layer activations

N data points in mini-batch

$$H' = \frac{H - \mu}{\sigma}$$

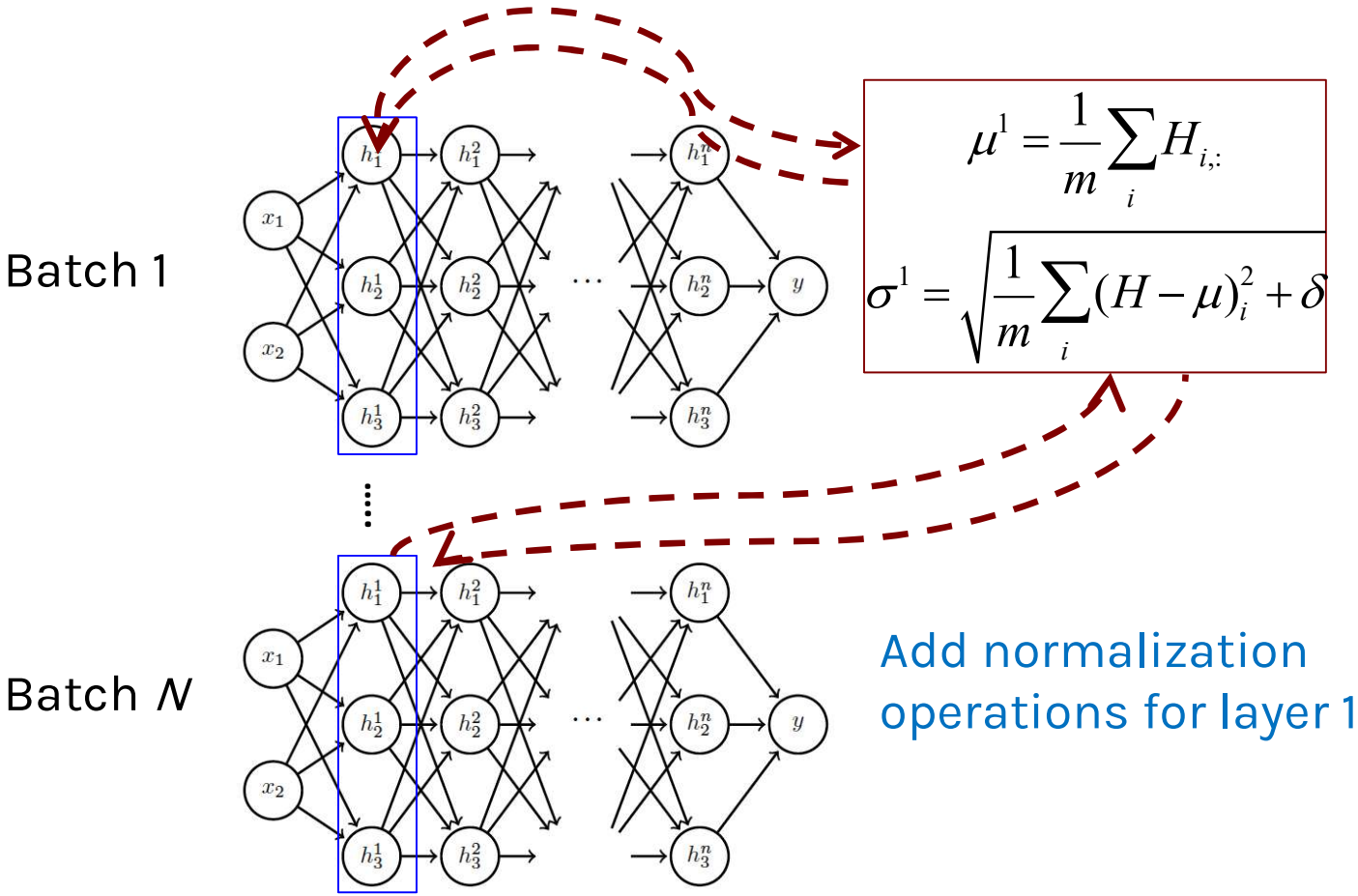
$$\mu = \frac{1}{m}\sum_i H_i \qquad \sigma = \sqrt{\frac{1}{m}\sum_i (H - \mu)_i^2 + \delta}$$

Vector of mean activations across mini-batch

Vector of SD of each unit across mini-batch

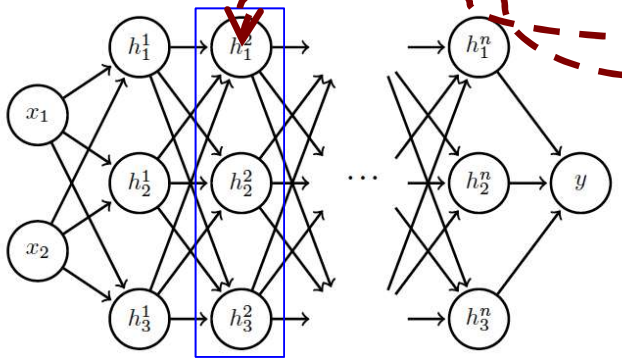
- Instead use: $\gamma H' + \beta$ — $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$ - learnable parameters
- Allows network to **control range of normalization**

Batch Normalization



Batch Normalization

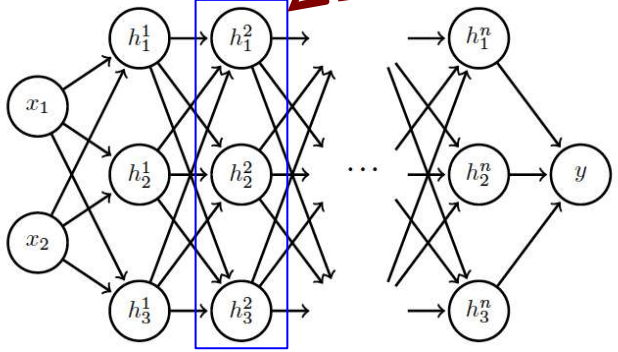
Batch 1



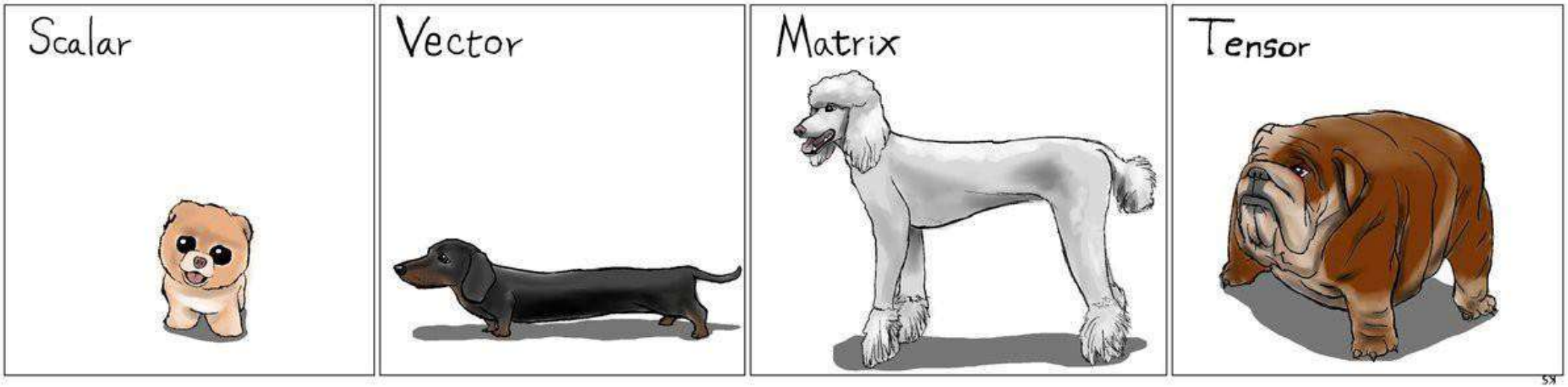
$$\mu^2 = \frac{1}{m}\sum_i H_{i,:}$$

$$\sigma^2 = \sqrt{\frac{1}{m}\sum_i (H-\mu)_i^2 + \delta}$$

Batch *N*

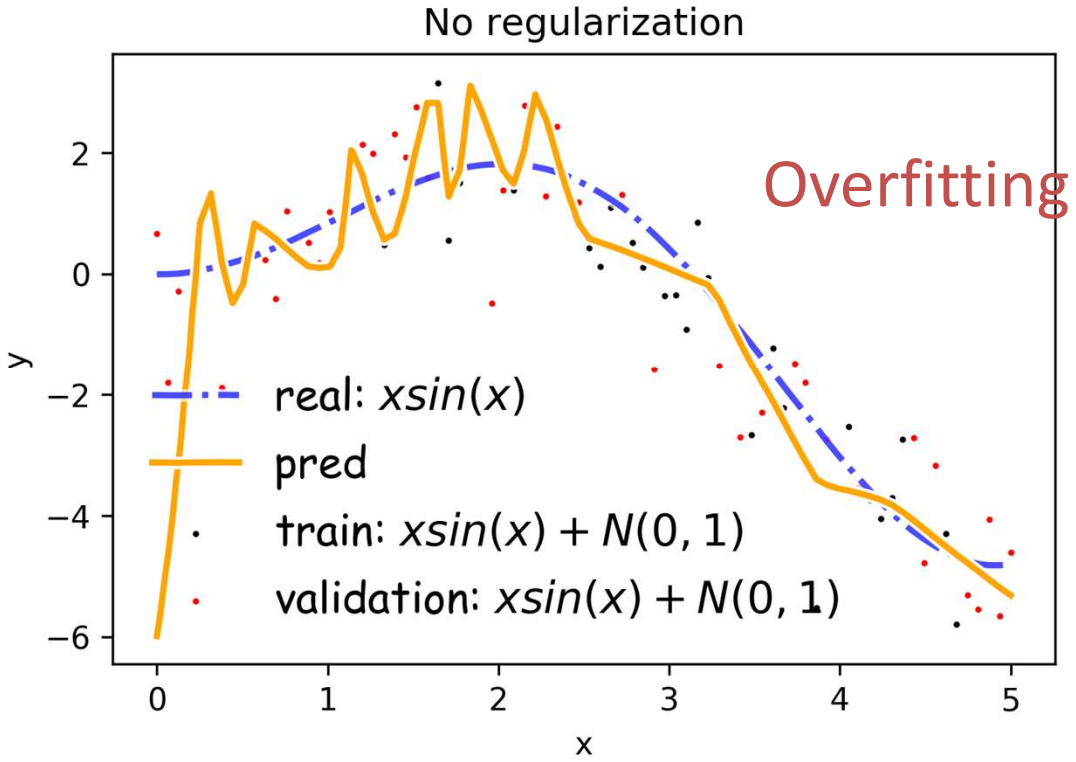


Add normalization operations for layer 2 and so on …


Scalar
Vector
Matrix
Tensor

Regularization

- Regularization is any modification we make to a learning algorithm that is intended to **reduce its generalization error** but not its training error.



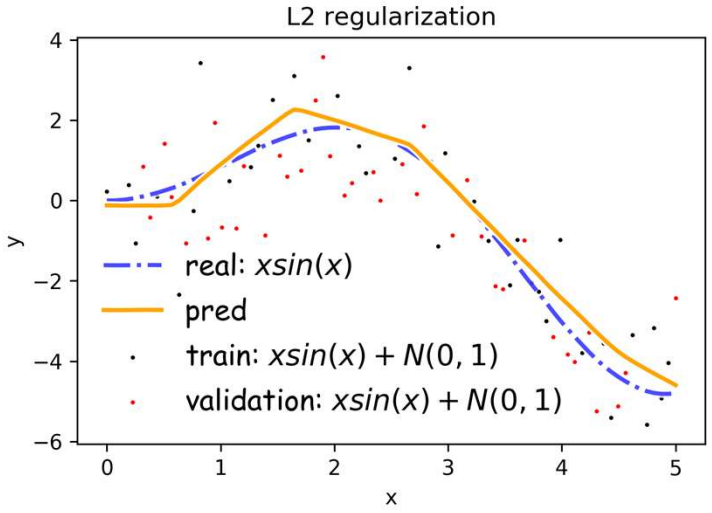
Fitting a deep neural network with 5 layers and 100 neurons per layer can lead to a very good prediction on the training set but poor prediction on validations set

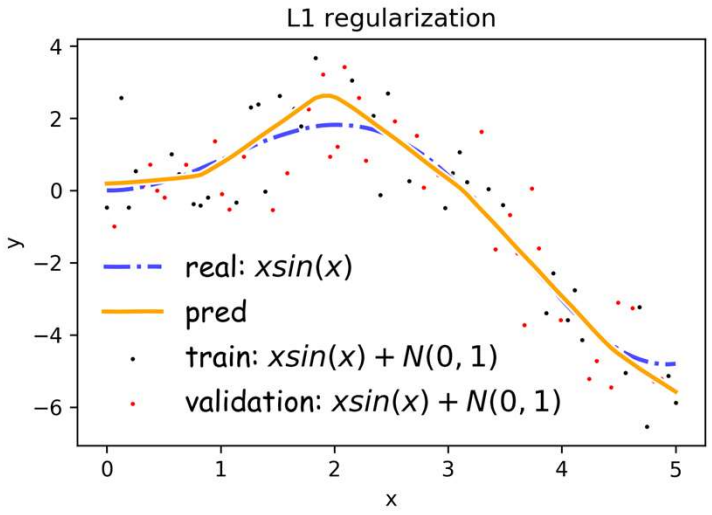
Norm Penalties

- We used to optimize: $\mathbf{L(W; X, y)}$
- Change to: $L_R(W; X, y) = L(W; X, y) + \boldsymbol{\alpha\Omega}(W)$

- L_2 regularization: $\Omega(W) = \frac{1}{2} \| W \|_2^2$
  - Weights decay
  - MAP estimation with Gaussian prior

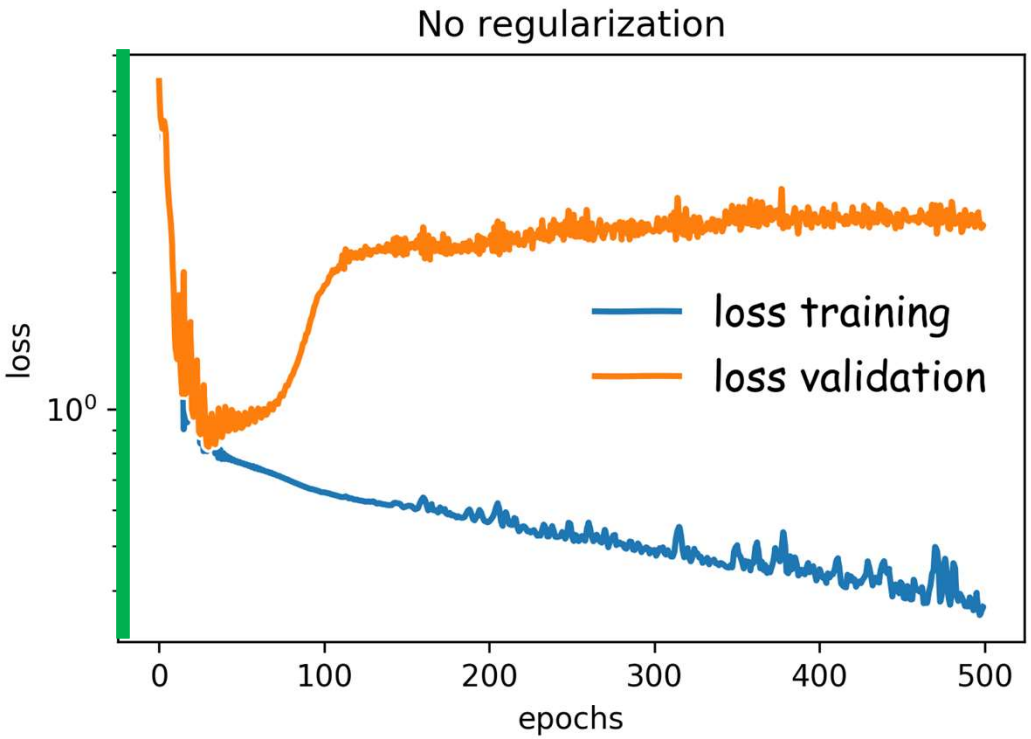
- L_1 regularization: $\Omega(W) = \frac{1}{2} \| W \|_1$
  - encourages sparsity
  - MAP estimation with Laplacian prior





Early Stopping

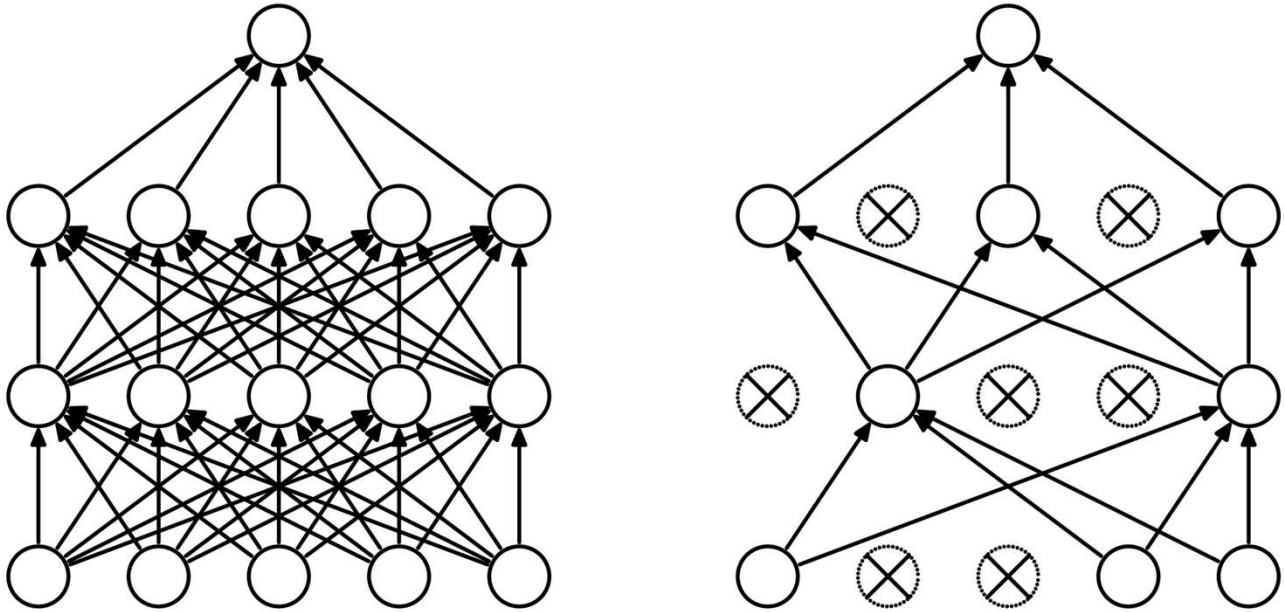
- Terminate while validation set performance is better



Training time can be treated as a hyperparameter too

Dropout

- Randomly set some neurons and their connections to zero (i.e. “dropped”)
- Prevent overfitting by reducing co-adaptation of neurons
- Like training many random sub-networks

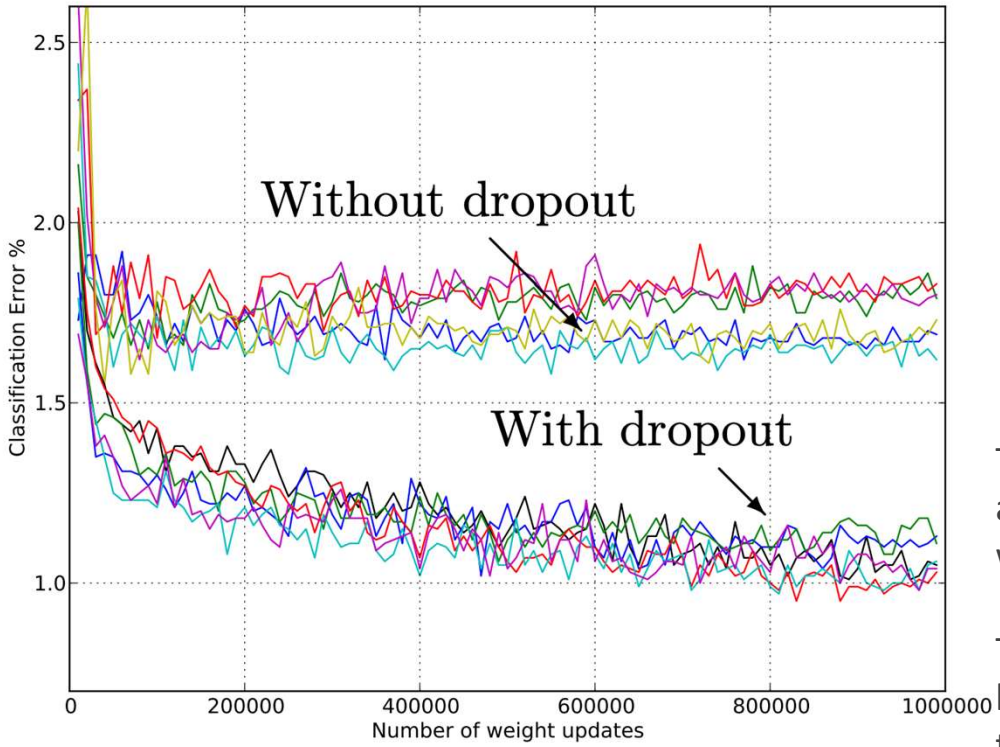


(a) Standard Neural Net

(b) After applying dropout.

Dropout

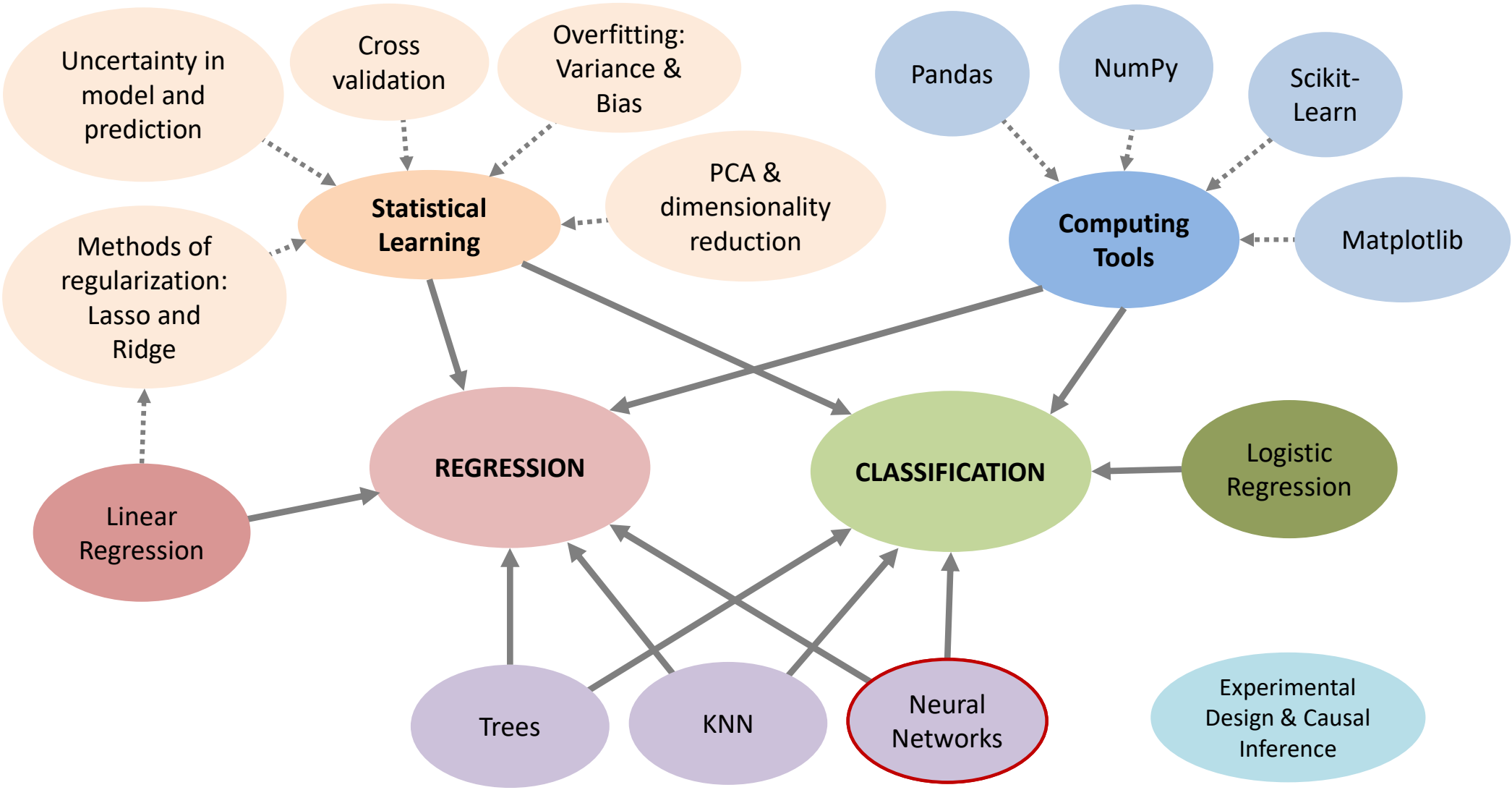
- Widely used and highly effective
- Proposed as an alternative to ensembling, which is too expensive for neural nets



Test error for different architectures with and without dropout.

The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf


Uncertainty in model and prediction
Cross validation
Overfitting: Variance & Bias
Pandas
NumPy
Scikit-Learn
Statistical Learning
PCA & dimensionality reduction
Computing Tools
Matplotlib
Methods of regularization: Lasso and Ridge
REGRESSION
CLASSIFICATION
Logistic Regression
Linear Regression
Trees
KNN
Neural Networks
Experimental Design & Causal Inference